

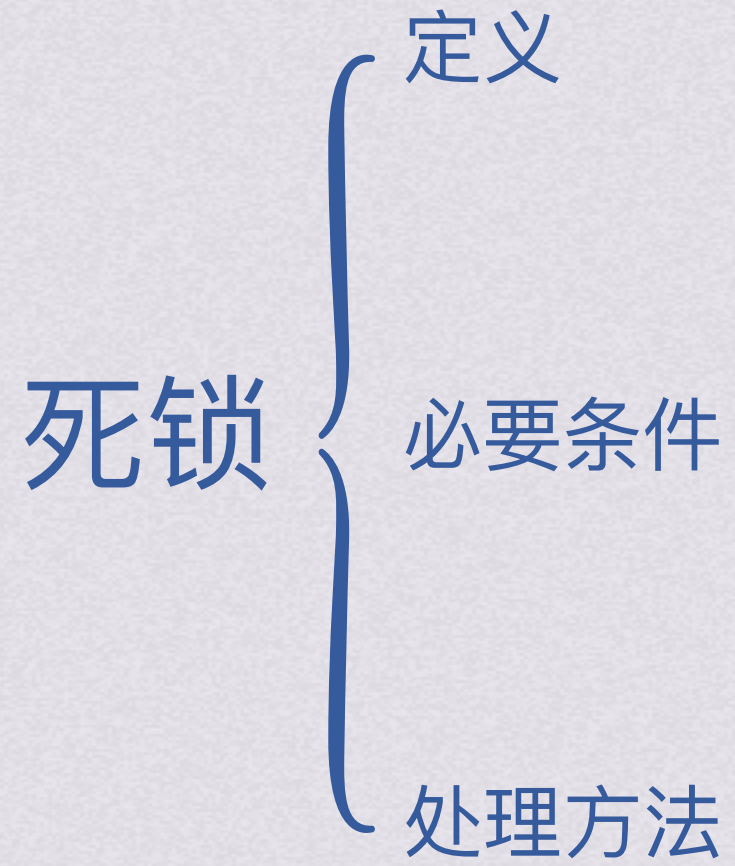
bok25

**基
礎**

課、操作系統



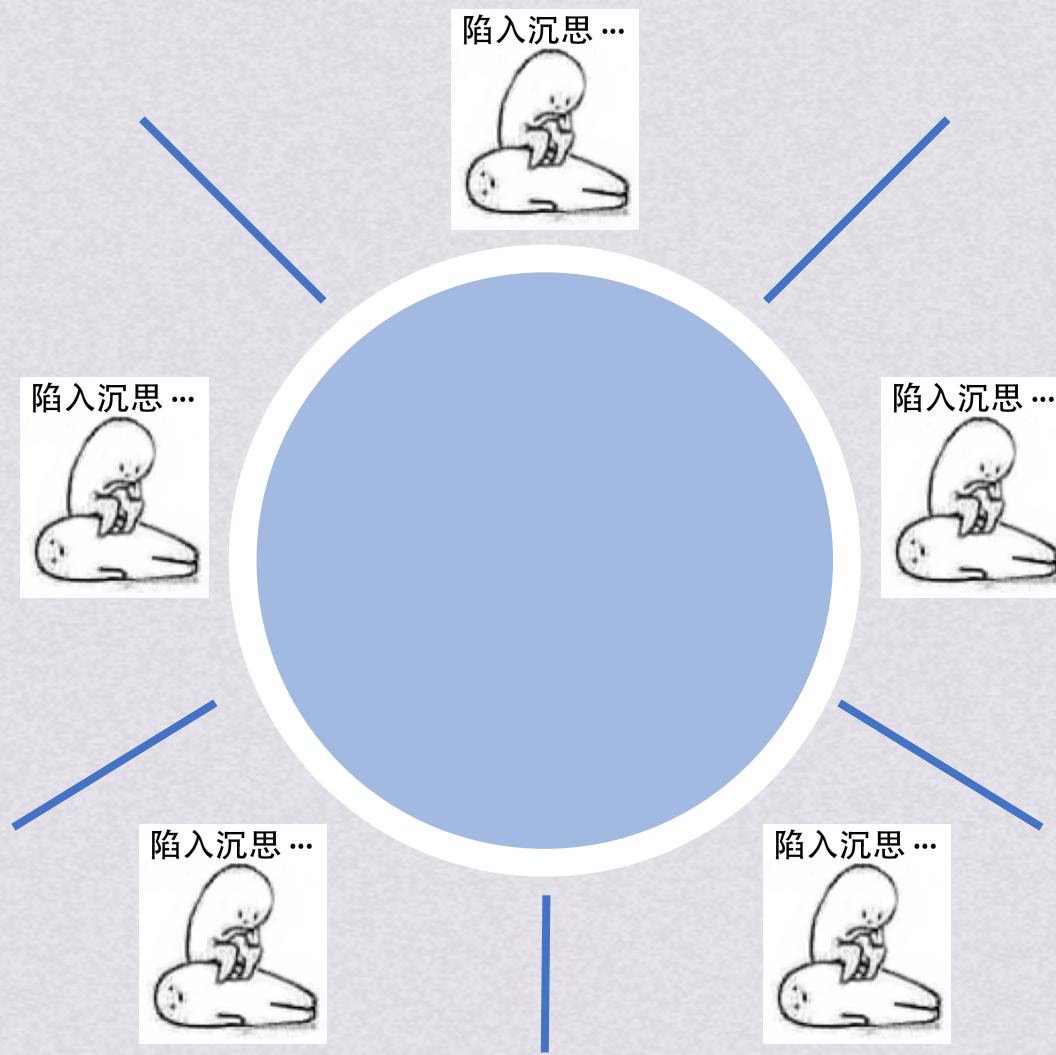
死锁





如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：

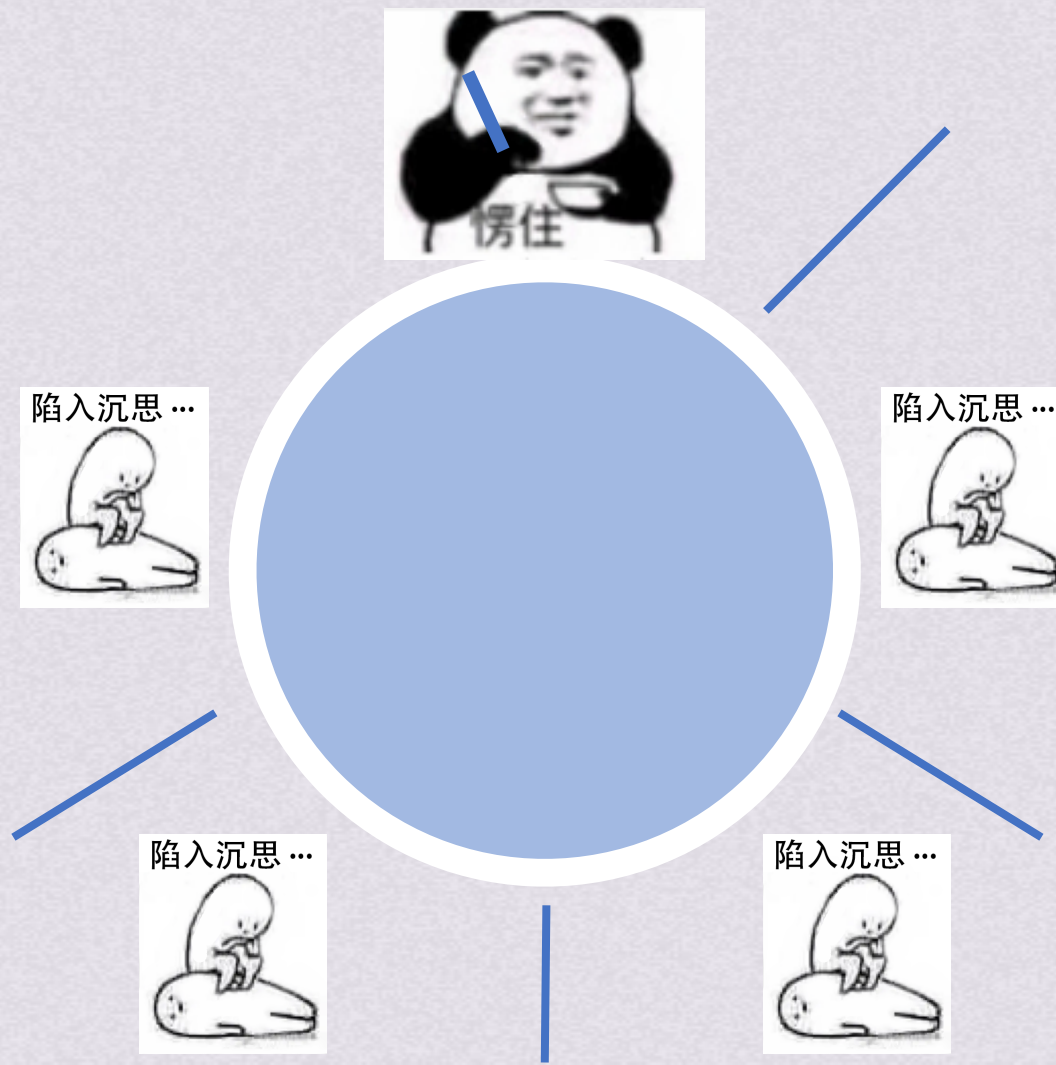


死锁 定义



如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

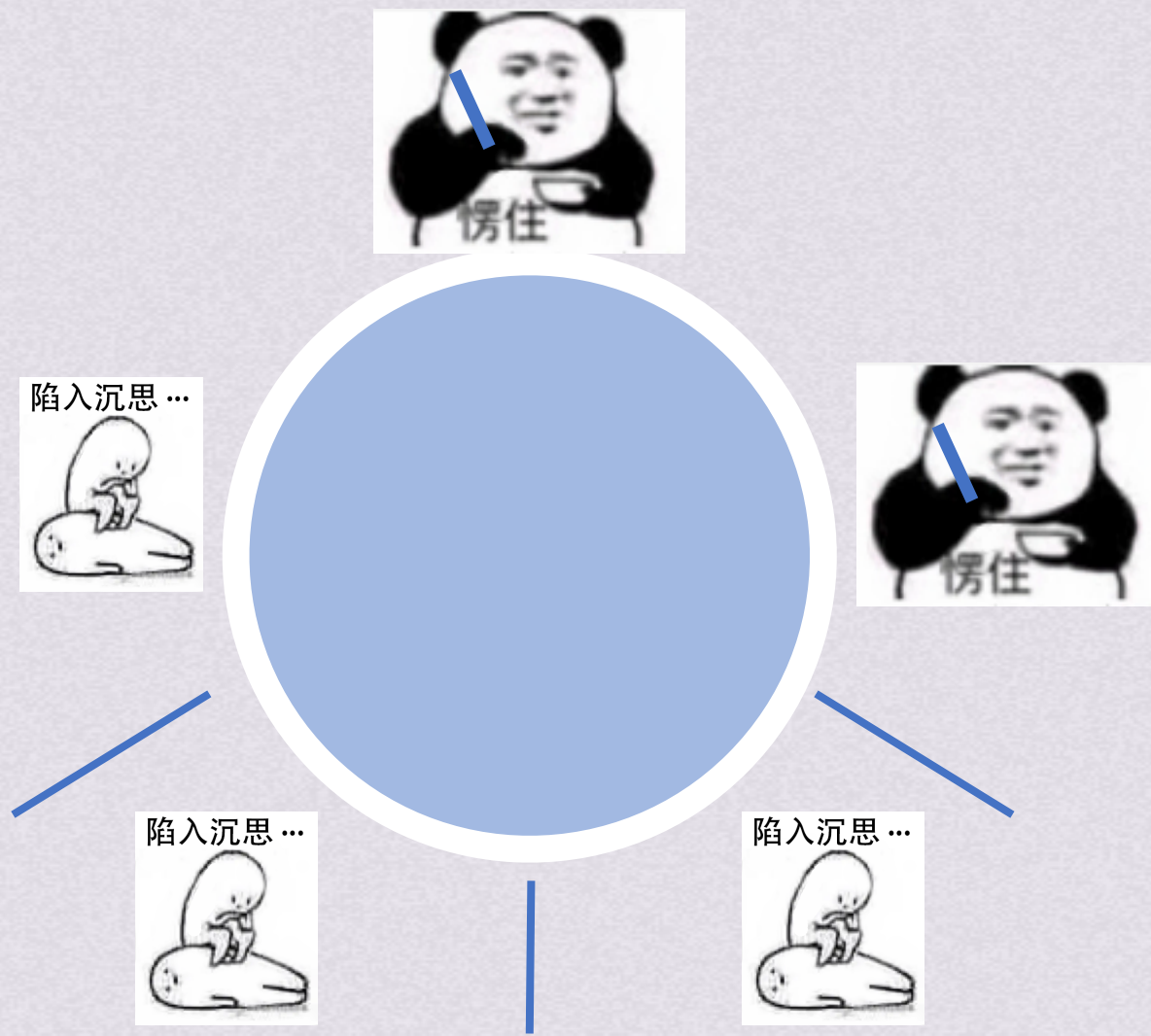
哲学家抢筷子的例子：





如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：

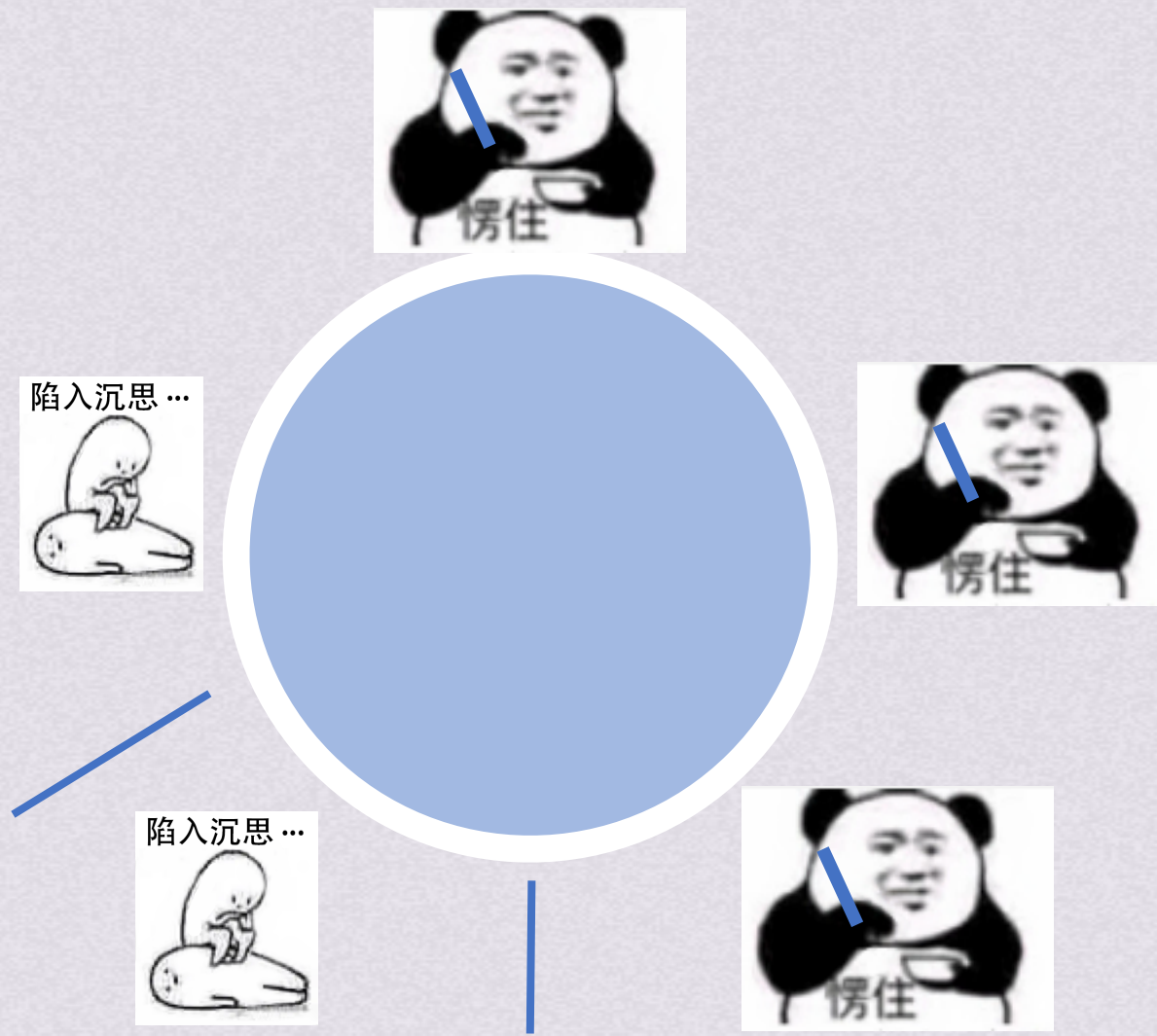


- 必要条件
- 处理方法



如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

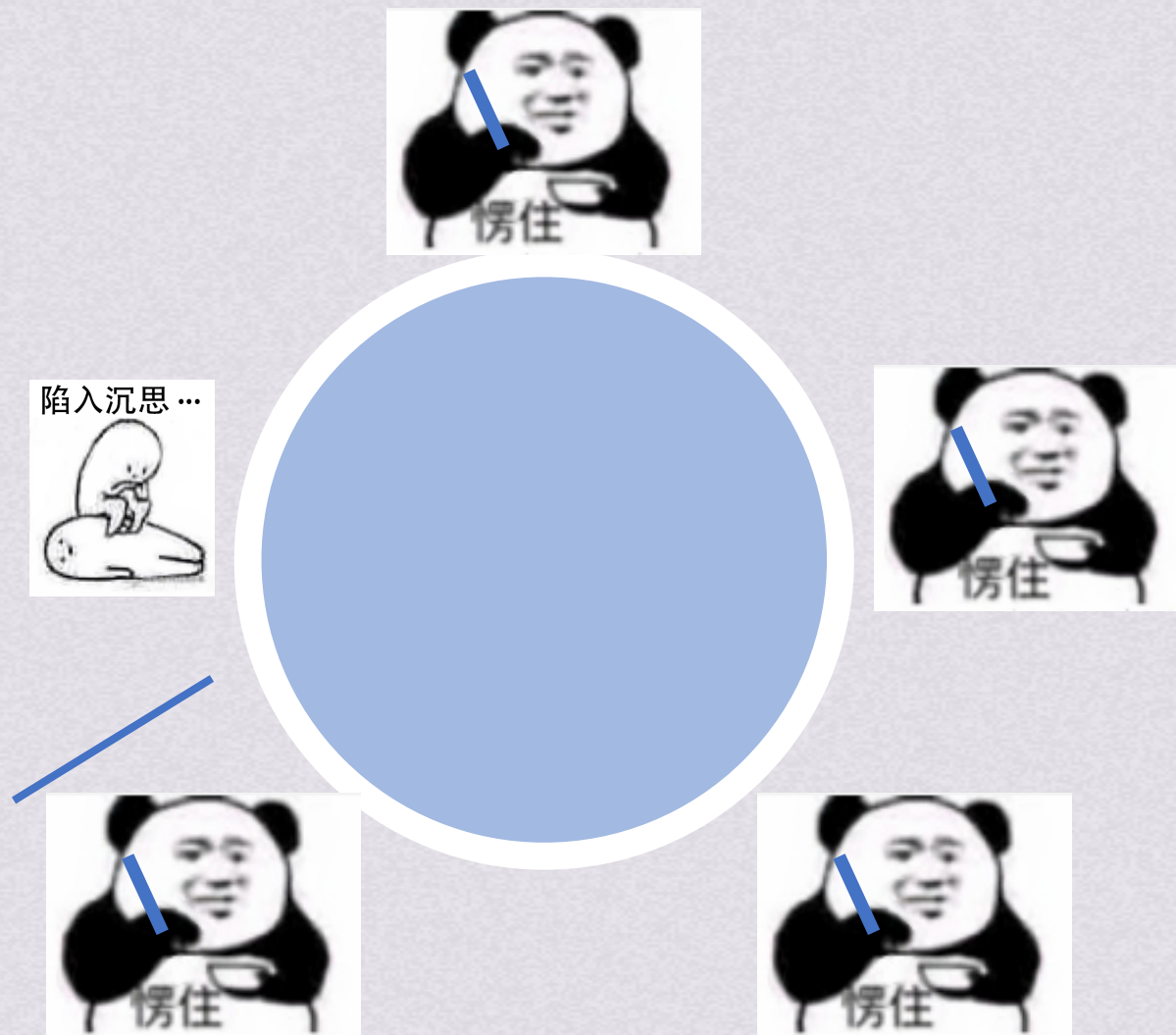
哲学家抢筷子的例子：





如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：



死锁 定义



如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

哲学家抢筷子的例子：

每个哲学家都拿着自己左边的筷子

都在等待右边的筷子

所有哲学家都不会主动释放自己左边的筷子

形成了死锁，所有人都没饭吃





如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的

饥饿：当等待时间(等CPU、等IO资源...)给进程的推进带来明显影响时，称发生了饥饿
饥饿不代表系统已经死锁，但至少有一进程执行被无限推迟

死锁产生的原因：

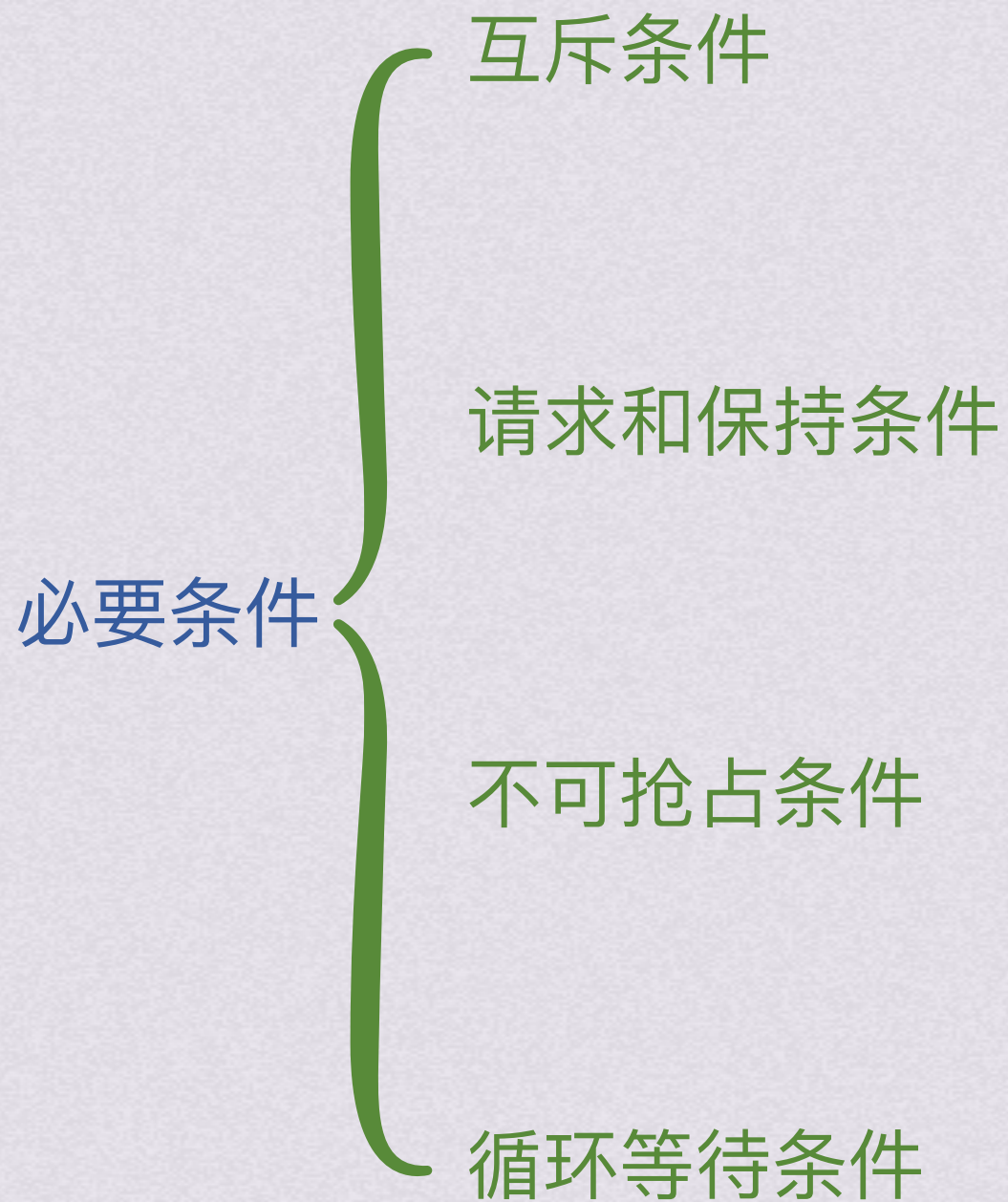
- 1.竞争不可抢占资源：系统中的不可抢占资源数量不足以满足多个进程运行需要，进程运行过程中就会因竞争资源而陷入僵局(对可抢占资源的竞争不会引起死锁)
- 2.进程推进顺序不当：哲学家进餐问题中，若事先规定好合法的吃饭顺序也不会引起死锁

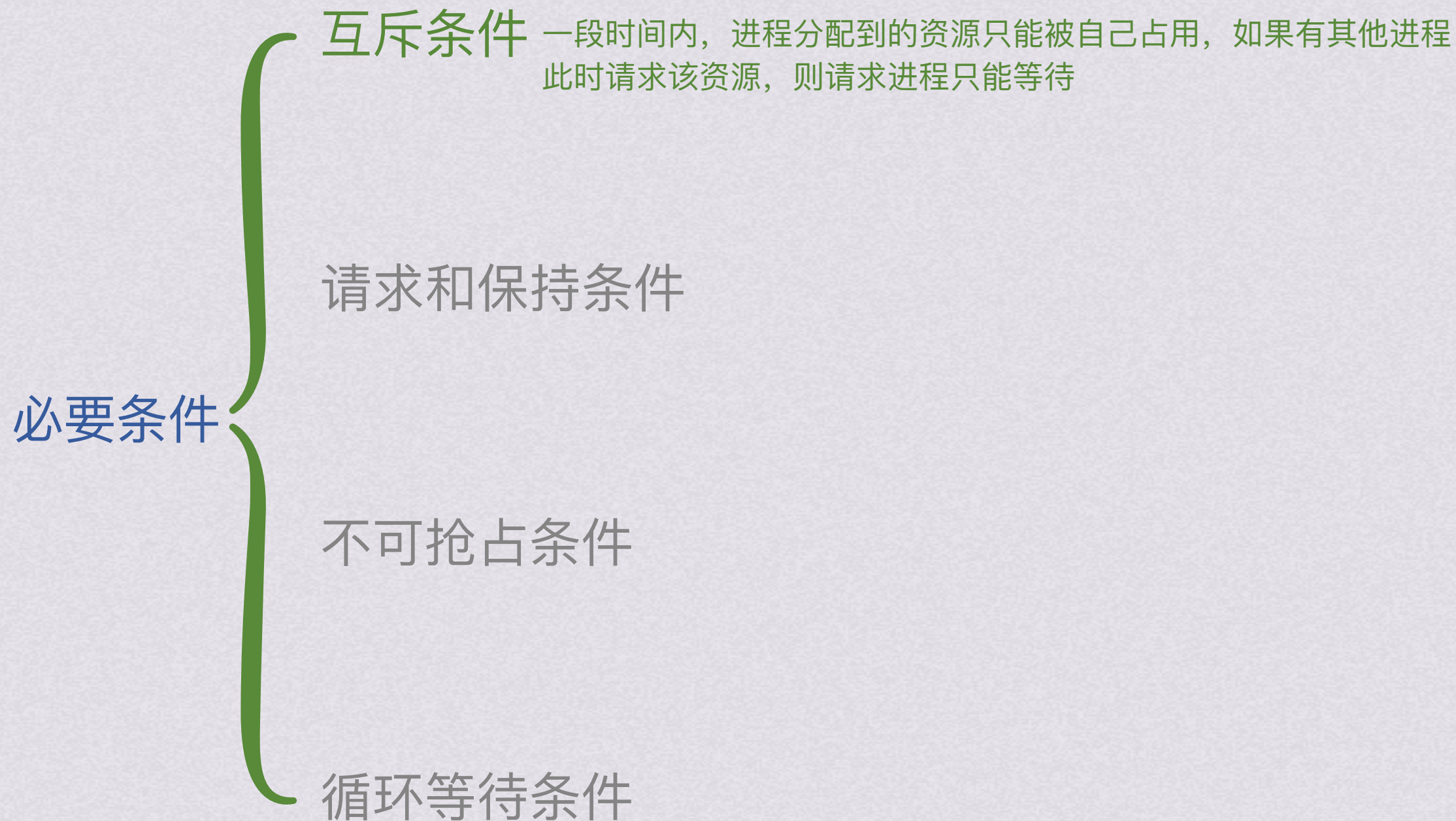


定义



必要条件







必要条件

互斥条件 一段时间内，进程分配到的资源只能被自己占用，如果有其他进程此时请求该资源，则请求进程只能等待

请求和保持条件

进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占有，此时请求进程被阻塞，但是对已有资源保持不放

不可抢占条件**循环等待条件**



必要条件

互斥条件 一段时间内，进程分配到的资源只能被自己占用，如果有其他进程此时请求该资源，则请求进程只能等待

请求和保持条件

进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占有，此时请求进程被阻塞，但是对已有资源保持不放

不可抢占条件

进程已获得的资源在未使用完之前不能被抢占，只能在进程使用完时由自己释放

循环等待条件



必要条件

互斥条件 一段时间内，进程分配到的资源只能被自己占用，如果有其他进程此时请求该资源，则请求进程只能等待

请求和保持条件

进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占有，此时请求进程被阻塞，但是对已有资源保持不放

不可抢占条件

进程已获得的资源在未使用完之前不能被抢占，只能在进程使用完时由自己释放

循环等待条件 所以系统出现死锁时必然会有多个进程处于阻塞态
发生死锁时必然存在一个进程——资源的循环链，P1等P2资源，P2等P3……



必要条件

互斥条件 一段时间内，进程分配到的资源只能被自己占用，如果有其他进程此时请求该资源，则请求进程只能等待

请求和保持条件

进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占有，此时请求进程被阻塞，但是对已有资源保持不放

不可抢占条件

进程已获得的资源在未使用完之前不能被抢占，只能在进程使用完时由自己释放

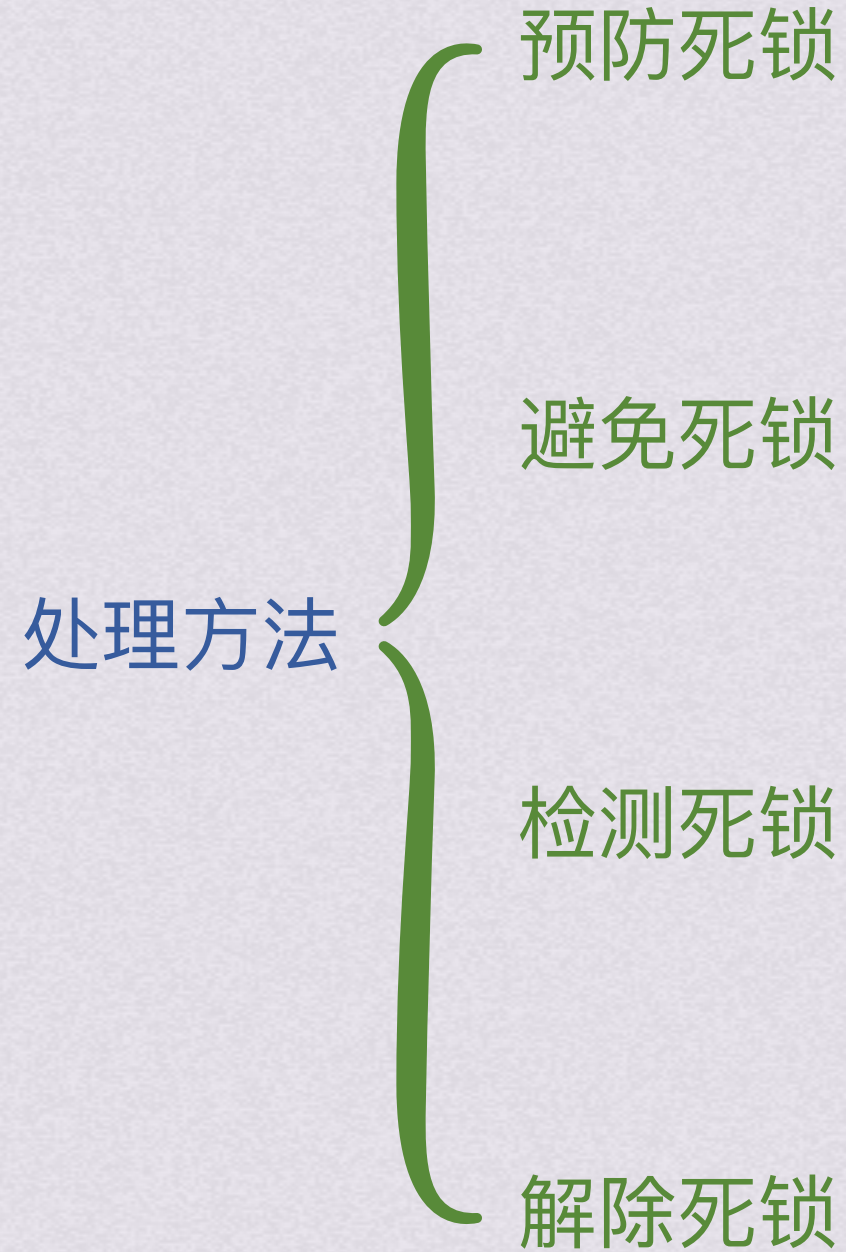
循环等待条件 所以系统出现死锁时必然会有多个进程处于阻塞态
发生死锁时必然存在一个进程——资源的循环链，P1等P2资源，P2等P3.....



必要条件



处理方法

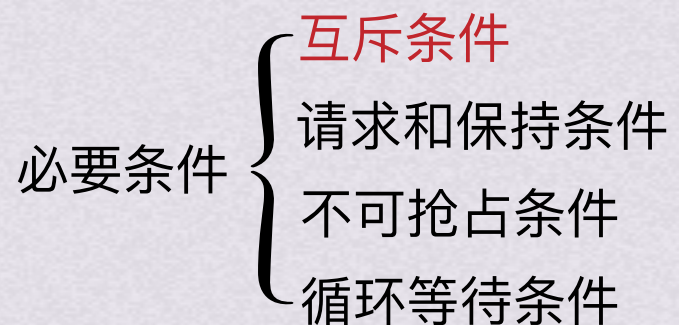




处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用



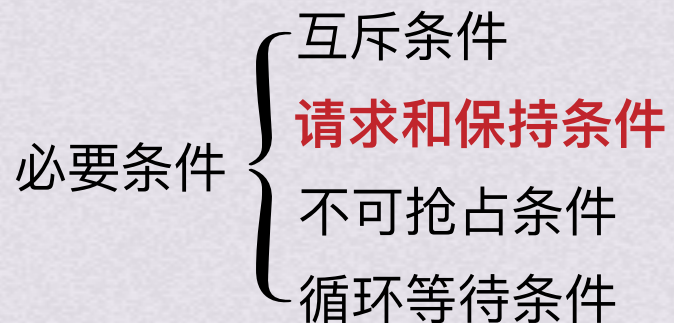
互斥条件不可被破坏，因为就是有临界资源只允许被互斥使用，我们还是来看看破坏别的吧



处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用



1.破坏请求和保持条件



处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

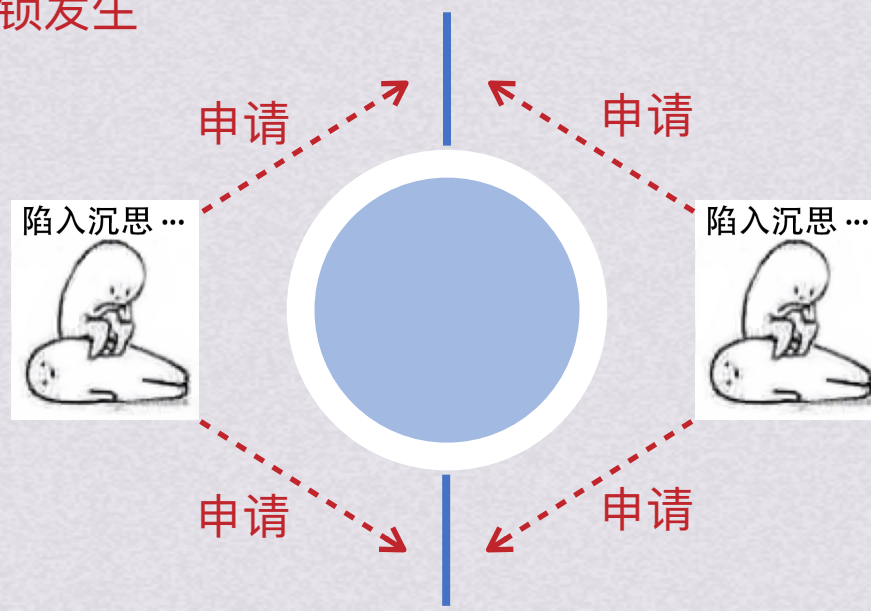
1.破坏请求和保持条件

可以让所有进程在开始运行之前，一次性地申请其在整个运行过程中需要的所有资源。若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时，只要有一种资源不能满足该进程要求，系统就不会给该进程其他任何资源，也破坏了“保持”条件，从而预防死锁发生

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件





处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

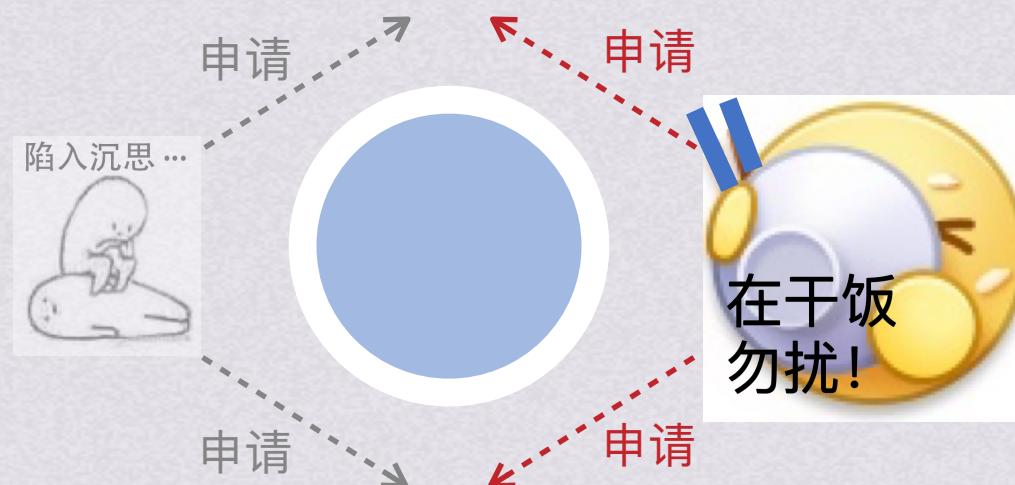
1.破坏请求和保持条件

可以让所有进程在开始运行之前，一次性地申请其在整个运行过程中需要的所有资源。若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时，只要有一种资源不能满足该进程要求，系统就不会给该进程其他任何资源，也破坏了“保持”条件，从而预防死锁发生

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件



避免死锁
检测死锁
解除死锁
定义
必要条件



处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

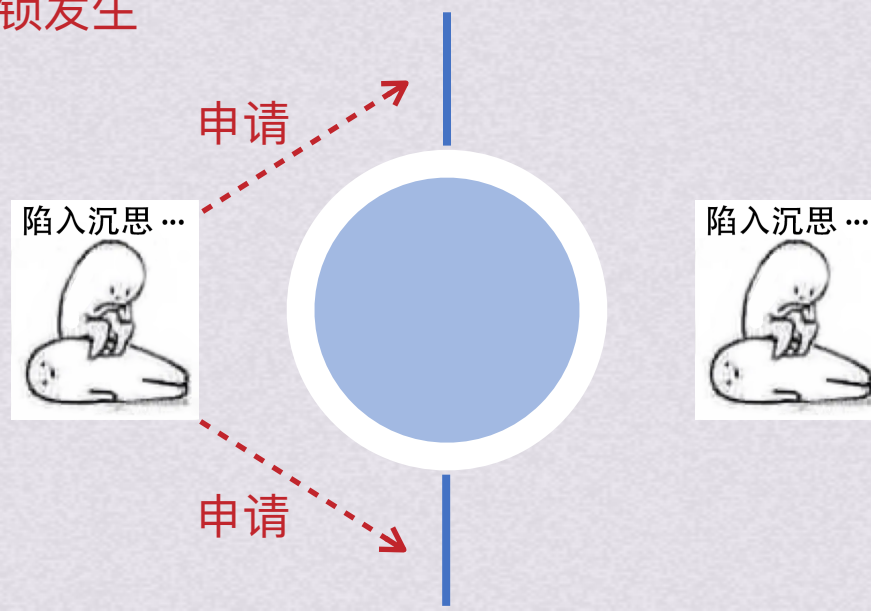
1.破坏请求和保持条件

可以让所有进程在开始运行之前，一次性地申请其在整个运行过程中需要的所有资源。若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时，只要有一种资源不能满足该进程要求，系统就不会给该进程其他任何资源，也破坏了“保持”条件，从而预防死锁发生

必要条件 {
互斥条件
请求和保持条件
不可抢占条件
循环等待条件





处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

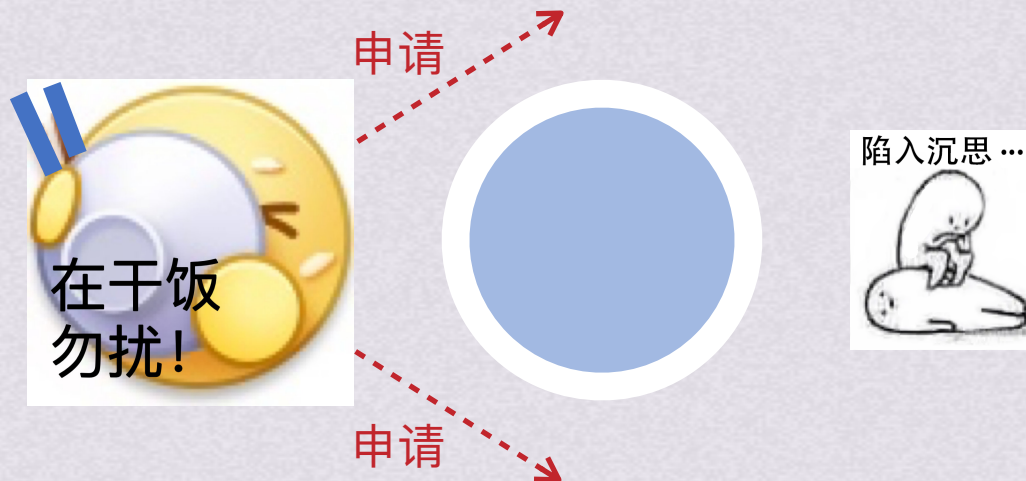
1.破坏请求和保持条件

可以让所有进程在开始运行之前，一次性地申请其在整个运行过程中需要的所有资源。若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时，只要有一种资源不能满足该进程要求，系统就不会给该进程其他任何资源，也破坏了“保持”条件，从而预防死锁发生

必要条件 {
互斥条件
请求和保持条件
不可抢占条件
循环等待条件





处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

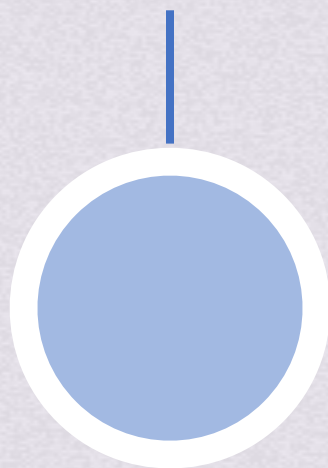
1.破坏请求和保持条件

可以让所有进程在开始运行之前，一次性地申请其在整个运行过程中需要的所有资源。若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时，只要有一种资源不能满足该进程要求，系统就不会给该进程其他任何资源，也破坏了“保持”条件，从而预防死锁发生

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件





处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

1.破坏请求和保持条件

可以让所有进程在开始运行之前，一次性地申请其在整个运行过程中需要的所有资源。若系统有足够的资源分给它，就给该进程所有它运行过程中需要的资源。

这样该进程在运行期间就不会在提出资源要求，破坏了“请求”条件

在申请资源时，只要有一种资源不能满足该进程要求，系统就不会给该进程其他任何资源，也破坏了“保持”条件，从而预防死锁发生

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件

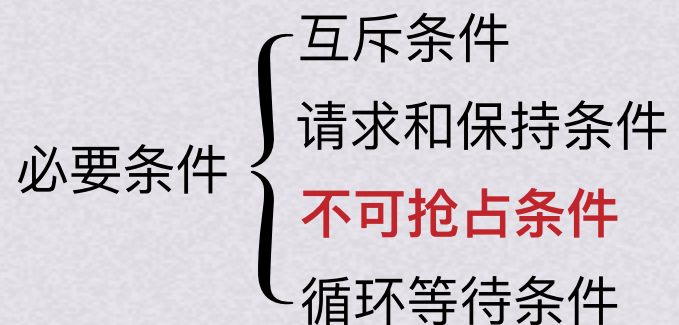
方法二：可以让进程申请，并分配给他初期运行时所需资源以方便进程运行，在进程释放出所有已分配资源以后才允许进程申请新的资源，这样能提高设备的利用率



处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用



2.破坏不可抢占条件



处理方法

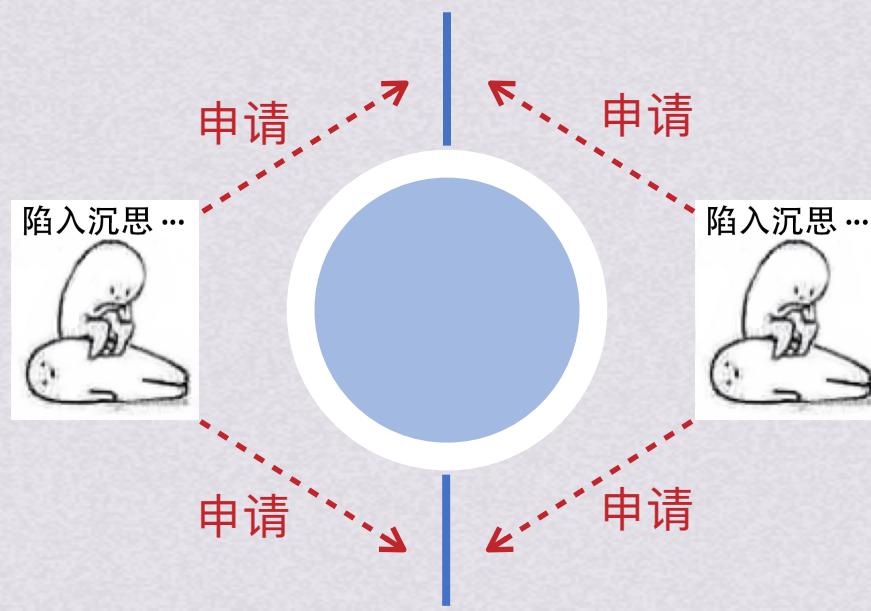
预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

2.破坏不可抢占条件

规定一个已经保持了某些不可被抢占资源的进程，提出新的资源请求不能被满足时，必须释放已经保持的所有资源，需要时再重新申请

必要条件 {
互斥条件
请求和保持条件
不可抢占条件
循环等待条件





处理方法

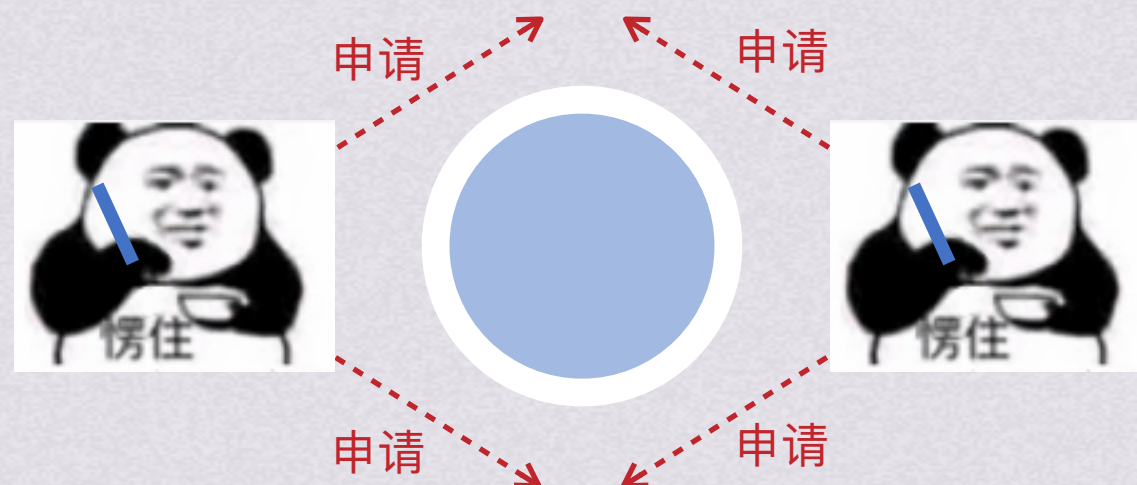
预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

2.破坏不可抢占条件

规定一个已经保持了某些不可被抢占资源的进程，提出新的资源请求不能被满足时，必须释放已经保持的所有资源，需要时再重新申请

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件



{ 避免死锁
检测死锁
解除死锁
定义
必要条件



处理方法

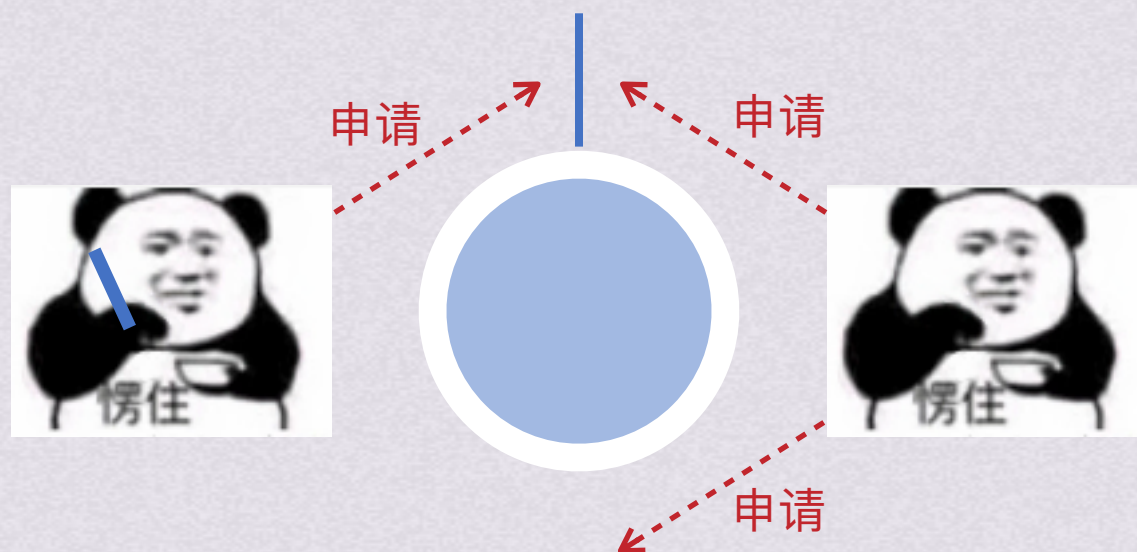
预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

2.破坏不可抢占条件

规定一个已经保持了某些不可被抢占资源的进程，提出新的资源请求不能被满足时，必须释放已经保持的所有资源，需要时再重新申请

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件



{ 避免死锁
检测死锁
解除死锁
定义
必要条件



处理方法

预防死锁

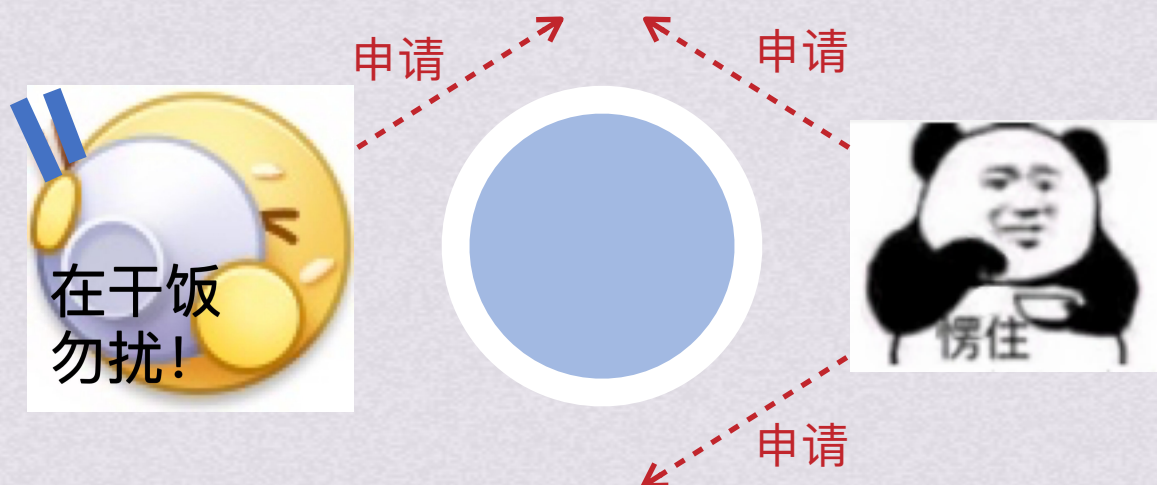
通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

2.破坏不可抢占条件

规定一个已经保持了某些不可被抢占资源的进程，提出新的资源请求不能被满足时，必须释放已经保持的所有资源，需要时再重新申请

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件

该方法实现复杂，且需付出很大代价



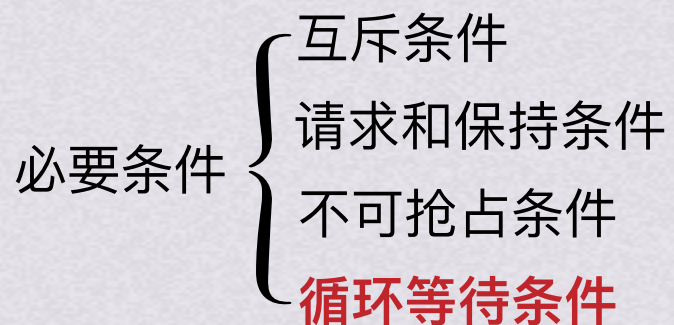
避免死锁
检测死锁
解除死锁
定义
必要条件



处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用



3.破坏循环等待条件



处理方法

预防死锁

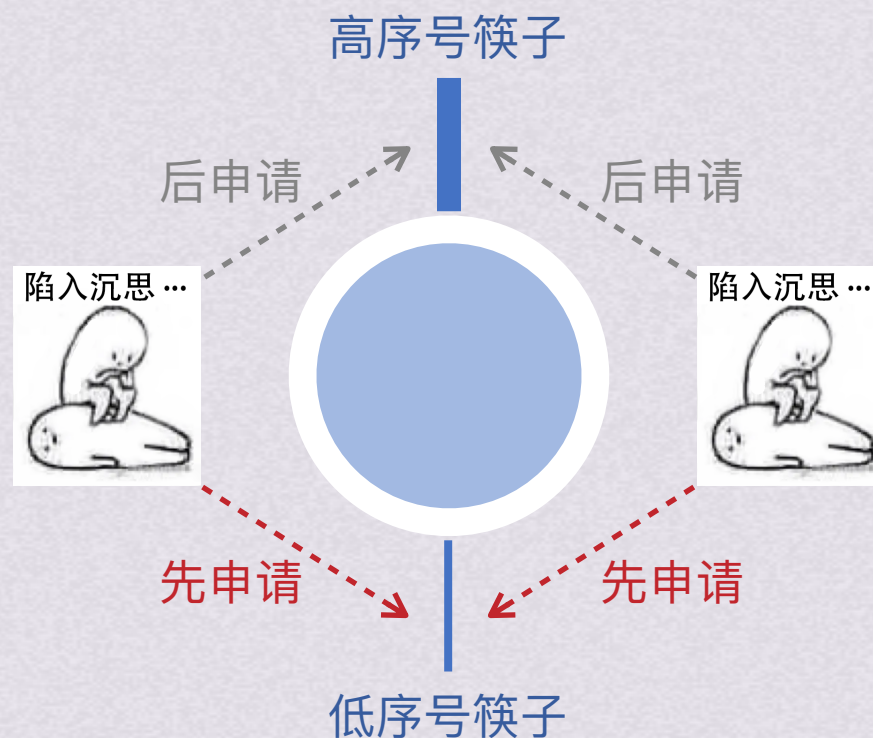
通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

3.破坏循环等待条件

对系统的资源进行线性排序，规定每个进程必须按照序号递增的顺序请求资源。若拥有高序号资源以后又想要低序号资源，则需要释放掉所有高序号资源，再申请低序号资源。这样就破坏了“循环等待”的条件

限制了申请资源的顺序

必要条件 { 互斥条件
请求和保持条件
不可抢占条件
循环等待条件



{ 避免死锁
检测死锁
解除死锁
定义
必要条件



处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

3.破坏循环等待条件

对系统的资源进行线性排序，规定每个进程必须按照序号递增的顺序请求资源。每个进程必须按照编号递增顺序申请资源，若拥有高序号资源以后又想要低序号资源，则需要释放掉所有高序号资源，再申请低序号资源。这样就破坏了“循环等待”的条件

限制了申请资源的顺序

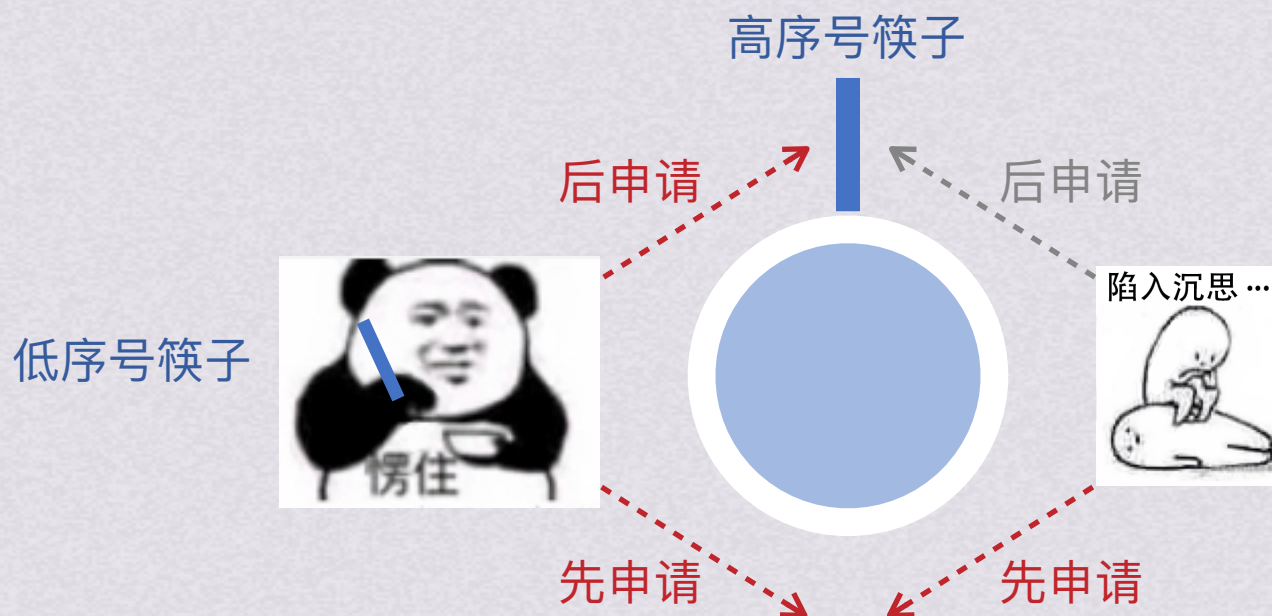
必要条件

互斥条件

请求和保持条件

不可抢占条件

循环等待条件





处理方法

预防死锁

通过设置某些限制条件，去破坏产生死锁四个必要条件中的一个或几个来预防产生死锁。预防死锁是一种比较容易实现的办法，已被广泛使用

3.破坏循环等待条件

对系统的资源进行线性排序，规定每个进程必须按照序号递增的顺序请求资源。每个进程必须按照编号递增顺序申请资源，若拥有高序号资源以后又想要低序号资源，则需要释放掉所有高序号资源，再申请低序号资源。这样就破坏了“循环等待”的条件

限制了申请资源的顺序

- 必要条件 {
- 互斥条件
 - 请求和保持条件
 - 不可抢占条件
 - 循环等待条件

这种方式因为需要为资源编号，就限制了新类型设备的增加；作业使用顺序与系统预期不同也会导致资源浪费





预防死锁



避免死锁



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序($P_1, P_2 \dots P_n$)称为安全序列

不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序($P_1, P_2 \dots P_n$)称为安全序列

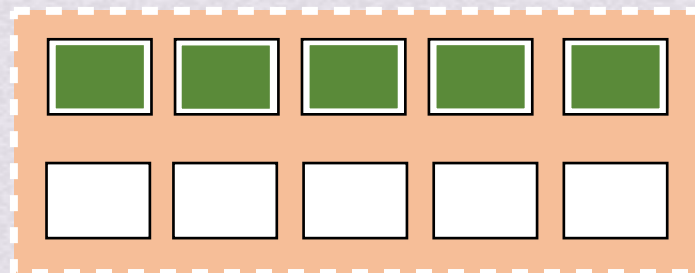
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

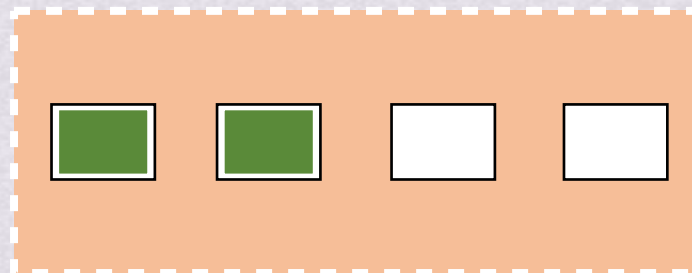
某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

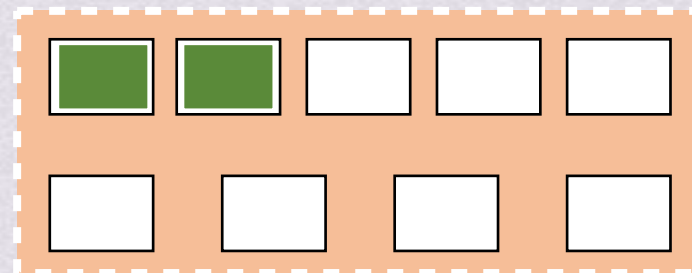
进程	资源需求 Need
P_1	5
P_2	2
P_3	7



P1



P2



P3

处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序(P1,P2...Pn)称为安全序列

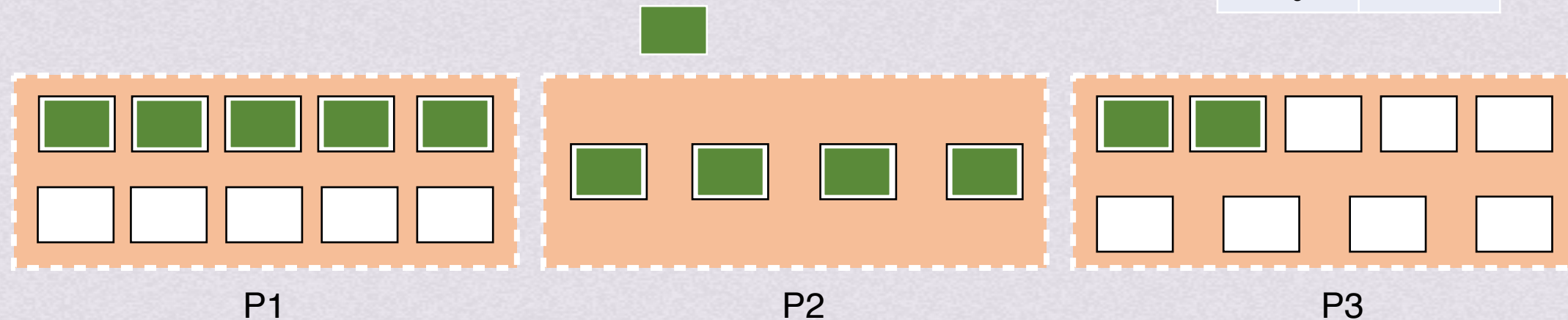
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	5
P ₂	0
P ₃	7





处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序(P1,P2...Pn)称为安全序列

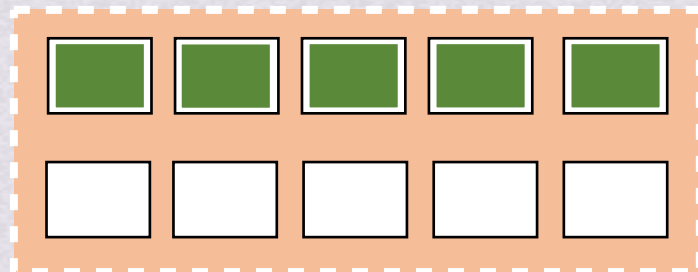
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

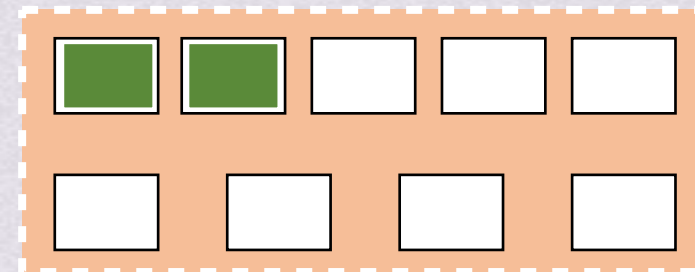
某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	5
P ₂	0
P ₃	7



P1



P3

P2

预防死锁
检测死锁
解除死锁
定义
必要条件



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序(P1,P2...Pn)称为安全序列

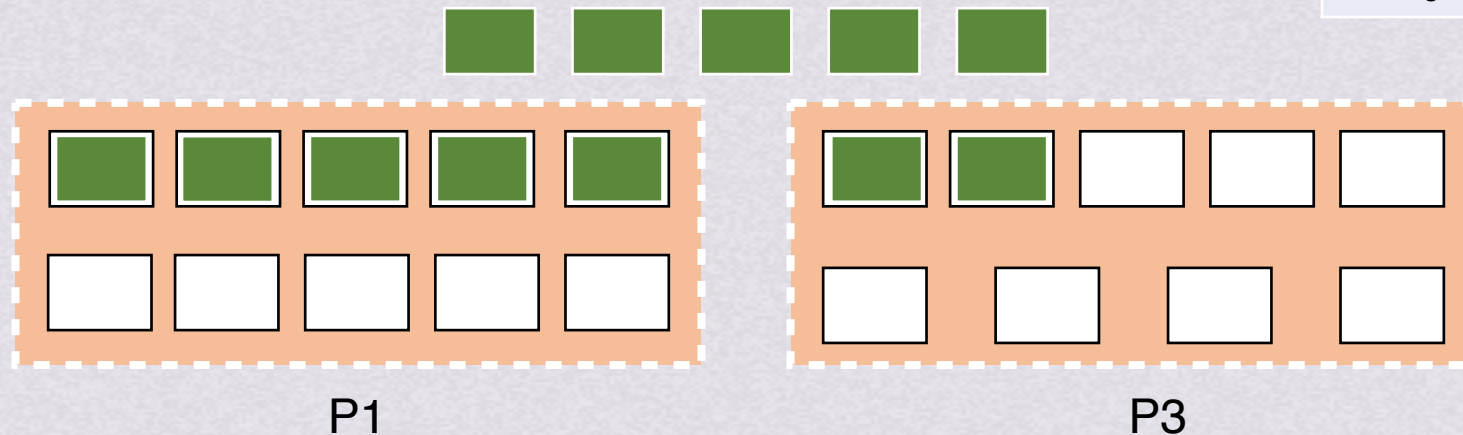
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	5
P ₂	0
P ₃	7





处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序($P_1, P_2 \dots P_n$)称为安全序列

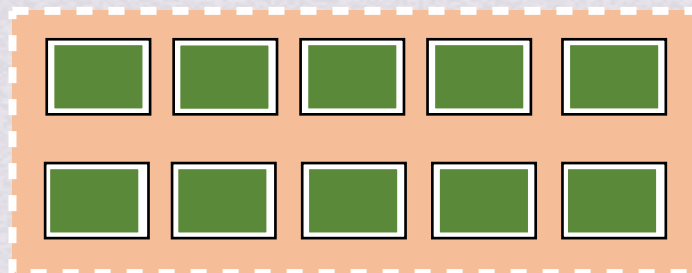
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

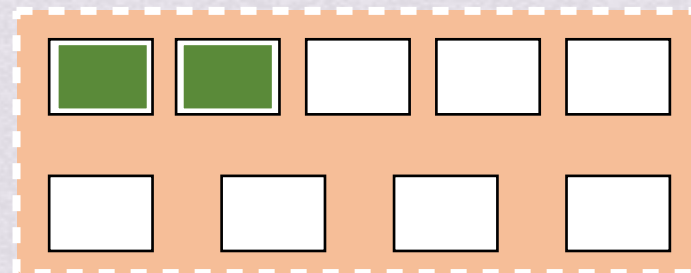
某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

进程	资源需求 Need
P_1	0
P_2	0
P_3	7



P_1



P_3



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序(P1,P2...Pn)称为安全序列

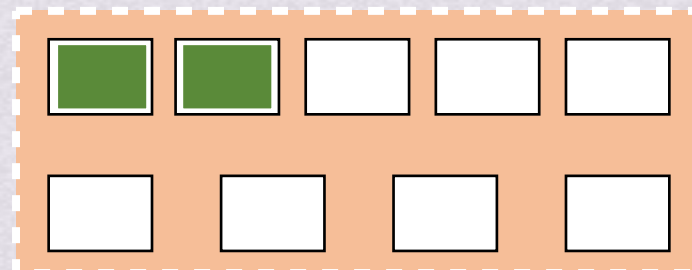
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P ₁	10	5	3
P ₂	4	2	
P ₃	9	2	

进程	资源需求 Need
P ₁	0
P ₂	0
P ₃	7



P3



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序($P_1, P_2 \dots P_n$)称为安全序列

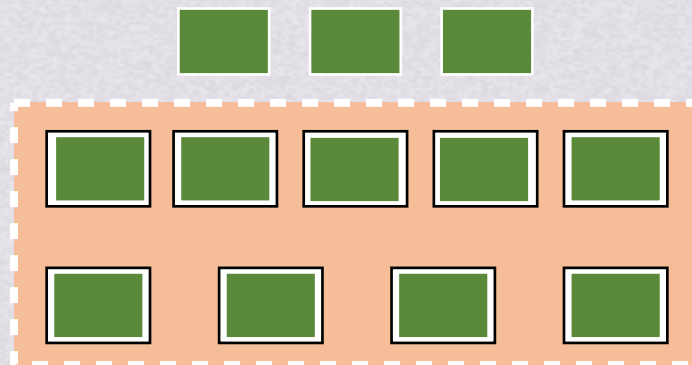
不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

进程	资源需求 Need
P_1	0
P_2	0
P_3	0



P_3



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

安全状态：如果系统能按某种进程推进顺序为每个进程分配其所需资源，满足每个进程对资源的最大需求，使每个进程都可顺利地完成。这样的进程推进顺序($P_1, P_2 \dots P_n$)称为安全序列

不安全状态：如果找不到安全序列，那系统此时是**不安全状态**。

进入**不安全状态**就可能发生死锁（也有可能不发生）；若系统处于**安全状态**，系统便不会进入死锁状态

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求	已分配	可用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

存在安全序列 $P_2 \rightarrow P_1 \rightarrow P_3$
系统处于安全状态

如果不按照安全序列执行的话会让系统进入不安全状态
不安全状态可能发生死锁

避免死锁

某时刻进程的资源使用情况见下表，求此刻的安全序列

进程	已分配资源			尚需分配			可用资源		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	2	0	0	0	0	1	0	2	1
P2	1	2	0	1	3	2			
P3	0	1	1	1	3	1			
P4	0	0	1	2	0	0			

可用资源此刻只能满足P1需求，分配给P1试试



避免死锁

某时刻进程的资源使用情况见下表，求此刻的安全序列

进程	已分配资源			尚需分配			可用资源		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	2	0	0	0	0	1	0	2	1
P2	1	2	0	1	3	2			
P3	0	1	1	1	3	1			
P4	0	0	1	2	0	0			

可用资源此刻只能满足P1需求，分配给P1试试

分配P1 → P1运行完成，释放资源

可用资源		
R1	R2	R3
2	2	1

只能满足P4资源，分配P4

→ P4运行完成，释放资源

可用资源		
R1	R2	R3
2	2	2

避免死锁

某时刻进程的资源使用情况见下表，求此刻的安全序列

进程	已分配资源			尚需分配			可用资源		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	2	0	0	0	0	1	0	2	1
P2	1	2	0	1	3	2			
P3	0	1	1	1	3	1			
P4	0	0	1	2	0	0			

可用资源		
R1	R2	R3
2	2	2

先后分配给P1， P4后，此时已有的资源无法满足任何进程，所以不存在安全序列
此时处于不安全状态

避免死锁
银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

可用资源:3
available

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
Need=Max-Allocation

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程发起申请，试图申请资源
请求向量被记录为Request

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

预防死锁
检测死锁
解除死锁
定义
必要条件



避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

可用资源:3
available

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
 $Need = Max - Allocation$

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

注意：本题只有一种资源，
所以j可以全部忽略
i为进程编号

若 $Request[i] > Need[i]$ 则出错，
因为所需资源已超过宣布的最大值

进程发起申请，试图申请资源
请求向量被记录为Request

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

可用资源:3
available

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
Need=Max-Allocation

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

注意：本题只有一种资源，
所以j可以全部忽略
i为进程编号

若Request[j] > Need[i,j]则出错，
因为所需资源已超过宣布的最大值
若Request[j] ≤ Need[i,j]，则计算下一步：
若Request[j] > Available[j]，该进程等待

进程发起申请，试图申请资源
请求向量被记录为Request

预防死锁
检测死锁
解除死锁
定义
必要条件

避免死锁 银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
 $Need = Max - Allocation$

可用资源:3
available

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	

P1需要等待

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

进程发起申请，试图申请资源
请求向量被记录为Request

注意：本题只有一种资源，
所以j可以全部忽略
i为进程编号

若 $Request[j] > Need[i,j]$ 则出错，
因为所需资源已超过宣布的最大值
若 $Request[j] \leq Need[i,j]$ ，则计算下一步：
若 $Request[j] > Available[j]$ ，该进程等待

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

可用资源:3
available

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
 $Need = Max - Allocation$

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

注意：本题只有一种资源，
所以j可以全部忽略
i为进程编号

若 $Request[j] > Need[i,j]$ 则出错，
因为所需资源已超过宣布的最大值
若 $Request[j] \leq Need[i,j]$ ，则计算下一步：
若 $Request[j] > Available[j]$ ，该进程等待

P2无需等待，系统尝试将资源分配给P2

进程发起申请，试图申请资源
请求向量被记录为Request

避免死锁
银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

注意：本题只有一种资源，
所以j可以全部忽略
i为进程编号

可用资源:3
available

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
Need=Max-Allocation

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

进程发起申请，试图申请资源
请求向量被记录为Request

若Request[j] > Need[i,j]则出错，
因为所需资源已超过宣布的最大值
若Request[j] ≤ Need[i,j]，则计算下一步：
若Request[j] > Available[j]，该进程等待
若Request[j] ≤ Available[j]，
则系统尝试将资源分配给该进程，
并修改对应数据结构中的数值

Available=Available-Request;
Allocation[i,j]=Allocation[i,j]+Request[i,j];
Need[i,j]=Need[i,j]-Request[i,j];

P2无需等待，系统尝试将资源分配给P2

预防死锁
检测死锁
解除死锁
定义
必要条件

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
 $Need = Max - Allocation$

可用资源:3 available	
进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

进程发起申请，试图申请资源
请求向量被记录为Request

若 $Request[j] > Need[i,j]$ 则出错，
因为所需资源已超过宣布的最大值
若 $Request[j] \leq Need[i,j]$ ，则计算下一步：
若 $Request[j] > Available[j]$ ，该进程等待
若 $Request[j] \leq Available[j]$ ，
则系统尝试将资源分配给该进程，
并修改对应数据结构中的数值

$Available = Available - Request;$
 $Allocation[i,j] = Allocation[i,j] + Request[j];$
 $Need[i,j] = Need[i,j] - Request[j];$

$Available = 3 - 2 = 1$
 $Allocation[2] = 2 + 2 = 4$
 $Need[2] = 2 - 2 = 0$



避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

- 1.设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
- 2.从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
- 3.进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]再返回步骤2
- 4.若安全序列中已有所有进程，则系统处于安全状态

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available

Work:3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4

Work:3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
3. 进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]
再返回步骤2

Work=3+2=5

Work:3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7



避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
3. 进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]再返回步骤2

Work:5

安全序列：P2

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available

2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4

Work:5

安全序列：P2

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available

2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4

Work:5

安全序列：P2

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

- 1.设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
- 2.从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
- 3.进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]
再返回步骤2

Work=5+5=10

Work:5

安全序列：P2->P1

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
3. 进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]再返回步骤2

Work:10

安全序列：P2->P1

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1.设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available

2.从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4

Work:10

安全序列：P2->P1

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1.设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available

2.从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4

Work:10

安全序列：P2->P1->P3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
3. 进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]
再返回步骤2

Work=10+2=12

Work:10

安全序列：P2->P1->P3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
3. 进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]
再返回步骤2

Work:12

安全序列：P2->P1->P3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

1.设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available

2.从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4

Work:12

安全序列：P2->P1->P3

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7



避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

Available = 1

Allocation[2] = 4

Need[2] = 0

带着这一组数据，执行安全性算法

- 1.设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令Work=Available
- 2.从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq$ work向量，找到后加入安全序列，找不到执行4
- 3.进程Pi进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算Work=Work+Allocation[i]再返回步骤2
- 4.若安全序列中已有所有进程，则系统处于安全状态

安全序列：P2->P1->P3

系统处于安全状态

Work:12

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7



避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

$Available = 1$

$Allocation[2] = 4$

$Need[2] = 0$

带着这一组数据，执行安全性算法

1. 设置工作向量Work表示系统中剩余可用资源数目，在执行算法前，令 $Work = Available$
2. 从Need矩阵中找出没加入安全序列中的进程，且该行 $\leq work$ 向量，找到后加入安全序列，找不到执行4
3. 进程 P_i 进入安全序列后，可顺利执行至完成并释放分配给它的资源，所以计算 $Work = Work + Allocation[i]$ 再返回步骤2
4. 若安全序列中已有所有进程，则系统处于安全状态

安全序列：P2->P1->P3

系统处于安全状态

Work:12

避免死锁

银行家算法计算过程

某资源T0时刻资源分配情况如图所示，现在系统是安全状态吗？

进程	最大需求 Max	已分配 Allocation
P ₁	10	5
P ₂	4	2
P ₃	9	2

计算需求(Need)矩阵
 $Need = Max - Allocation$

可用资源:3
available

进程	需求矩阵 Need
P ₁	5
P ₂	2
P ₃	7

进程	请求向量 Request
P ₁	5
P ₂	2
P ₃	7

进程发起申请，试图申请资源
请求向量被记录为Request

若 $Request[j] > Need[i,j]$ 则出错，
因为所需资源已超过宣布的最大值
若 $Request[j] \leq Need[i,j]$ ，则计算下一步：
若 $Request[j] > Available[j]$ ，该进程等待
若 $Request[j] \leq Available[j]$ ，
则系统尝试将资源分配给该进程，
并修改对应数据结构中的数值

$Available = Available - Request;$
 $Allocation[i,j] = Allocation[i,j] + Request[j];$
 $Need[i,j] = Need[i,j] - Request[j];$

$Available = 3 - 2 = 1$
 $Allocation[2] = 2 + 2 = 4$
 $Need[2] = 2 - 2 = 0$

安全序列：P2->P1->P3

系统处于安全状态

此次分配安全，把资源分配给P2



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待





处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构

可利用资源向量Available

一个含有m个元素的数组，记录m类可利用资源的数目，数值随着资源的分配回收动态改变

最大需求矩阵Max

分配矩阵Allocation

需求矩阵Need



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构

可利用资源向量Available

一个含有m个元素的数组，记录m类可利用资源的数目，数值随着资源的分配回收动态改变

最大需求矩阵Max

一个 $n*m$ 的矩阵，定义了系统中n个进程每个进程对m类资源的最大需求

分配矩阵Allocation

需求矩阵Need



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构

可利用资源向量Available

一个含有m个元素的数组，记录m类可利用资源的数目，数值随着资源的分配回收动态改变

最大需求矩阵Max

一个n*m的矩阵，定义了系统中n个进程每个进程对m类资源的最大需求

分配矩阵Allocation

一个n*m的矩阵，记录了系统中每一类资源当前已分配给每一进程的资源数

需求矩阵Need



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构

可利用资源向量Available

一个含有m个元素的数组，记录m类可利用资源的数目，数值随着资源的分配回收动态改变

最大需求矩阵Max

一个n*m的矩阵，定义了系统中n个进程每个进程对m类资源的最大需求

分配矩阵Allocation

一个n*m的矩阵，记录了系统中每一类资源当前已分配给每一进程的资源数

需求矩阵Need

一个n*m的矩阵，表示每一个进程还需要的各类资源数



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构

可利用资源向量Available

一个含有m个元素的数组，记录m类可利用资源的数目，数值随着资源的分配回收动态改变

最大需求矩阵Max

一个n*m的矩阵，定义了系统中n个进程每个进程对m类资源的最大需求

分配矩阵Allocation

一个n*m的矩阵，记录了系统中每一类资源当前已分配给每一进程的资源数

需求矩阵Need

一个n*m的矩阵，表示每一个进程还需要的各类资源数

$$\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$$



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构 { 可利用资源向量Available
最大需求矩阵Max
分配矩阵Allocation
需求矩阵Need

进程传来请求向量 $Request_i$ ，若 $Request_i[j]=K$ ，表示进程 P_i 需要 K 个 R_j 类型的资源。系统按以下步骤检查：

1. 若 $Request_i[j] \leq Need[i,j]$ ，转向步骤2；否则认为出错，因为所需要的资源数超过所宣布最大值
2. 若 $Request_i[j] \leq Available[j]$ ，转向步骤；否则表示没有足够资源，进程需等待
3. 系统试探着把资源分给进程，并修改数值：

$$Available[j] = Available[j] - Request_i[j];$$

$$Allocation[i,j] = Allocation[i,j] + Request_i[j];$$

$$Need[i,j] = Need[i,j] - Request_i[j];$$

4. 系统执行安全性算法，检查此次资源分配后系统是否处于安全状态。若安全则正式分配；否则将本次试探分配作废，恢复原来资源分配状态，让进程等待



处理方法

避免死锁

避免死锁也是事先预防的策略，但是是在资源分配的过程中防止系统进入**不安全状态**，以避免发生死锁。这种方法限制条件少，系统性能好，目前常用这种方法避免发生死锁

银行家算法：最著名的死锁避免算法，每个新进程进入系统时，必须申明在运行过程中可能需要每种资源类型的最大单元数目，其数目不应超过系统所拥有的资源总量。进程请求一组资源时，系统确定是否有资源，分配后是否会导致系统处于不安全状态，如果不会，才分配资源，否则让进程等待

银行家算法中的数据结构 { 可利用资源向量Available
最大需求矩阵Max
分配矩阵Allocation
需求矩阵Need

安全性算法：

1.设置两向量：

工作向量Work：表示系统可提供给进程继续运行所需的各类资源数目，它含m个元素，执行安全算法开始时，
 $Work=Available$ ；

Finish:表示系统是否有足够的资源分配给进程，使之运行完成。开始时先做 $Finish[i]=false$;当有足够资源分配给进程时，再令 $Finish[i]=true$

2.从进程集合中找到一个能满足以下条件的进程：

$Finish[i] = false$;
 $Need[i,j] \leq Work[j]$;

找到则执行步骤3;
否则执行步骤4

3.进程 P_i 获得资源后，可顺利执行直至完成，并释放出分配给它的资源，应执行

$Work[j]=Work[j] + Allocation[i,j]$
 $Finish[i] = true$
去第二步

4.如果所有进程的 $Finish[i] = true$ 都满足，则表示系统处于安全状态，否则处于不安全状态



避免死锁



检测死锁

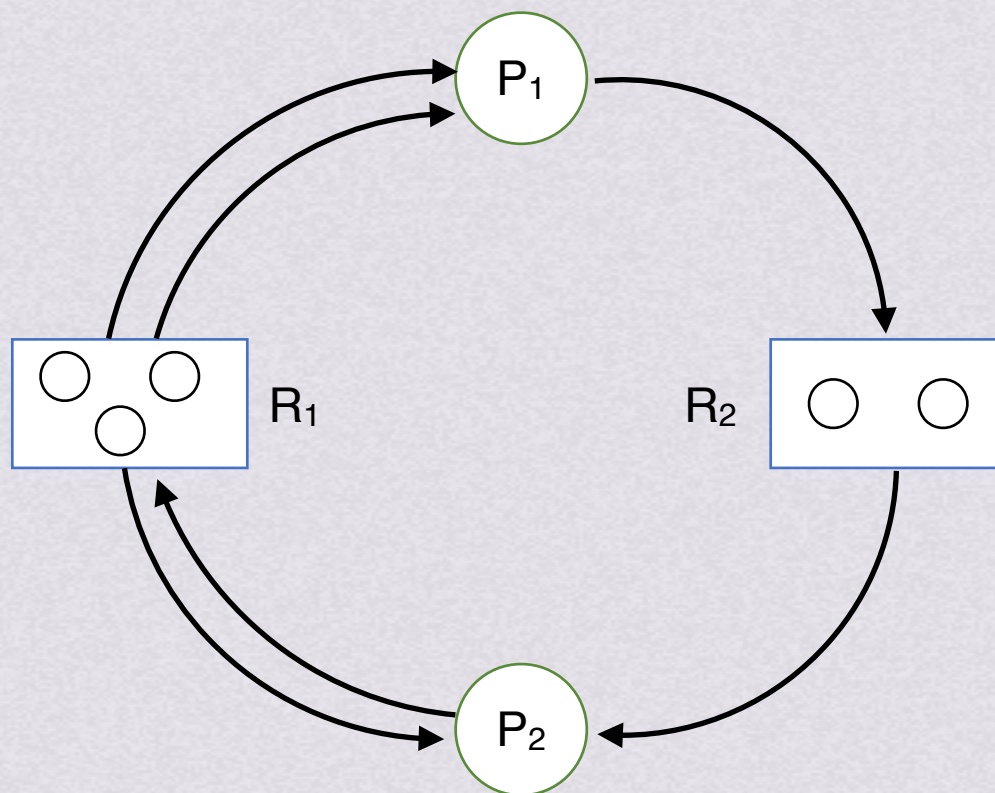


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

说人话：进程画圆圈，资源画方框里的圆圈



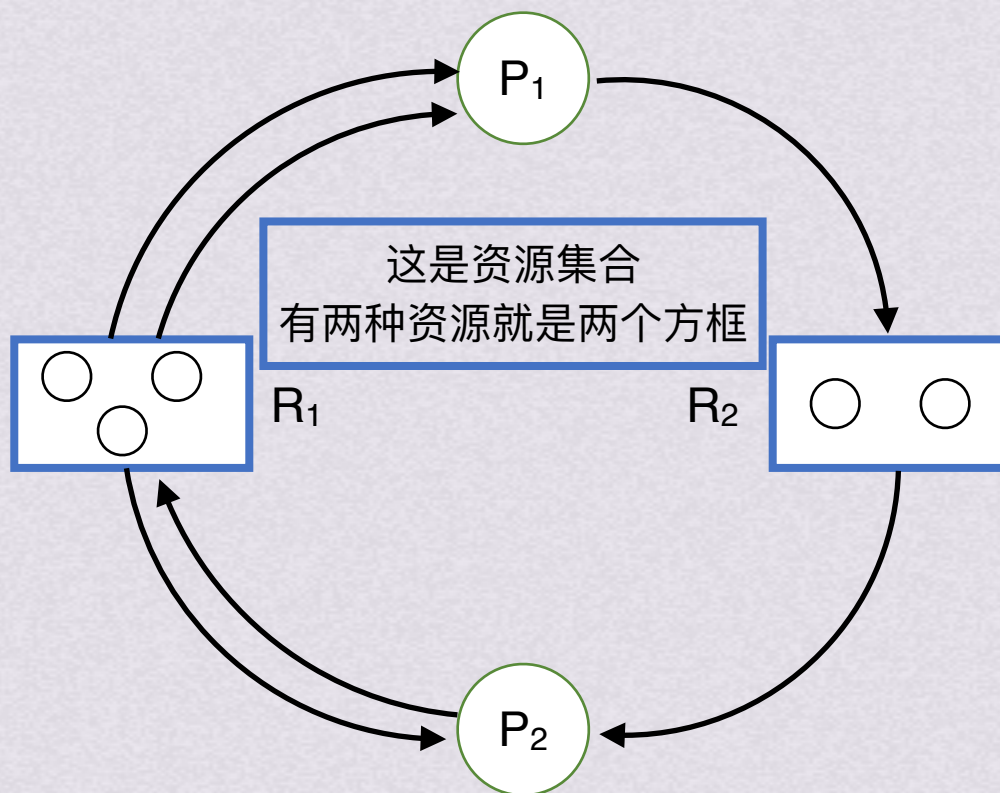


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点 N 和一组边 E 组成的一个对偶 $G=(N,E)$ 把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R ，每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边； R 指向 P 的是资源分配边

说人话：进程画圆圈，资源画方框里的圆圈



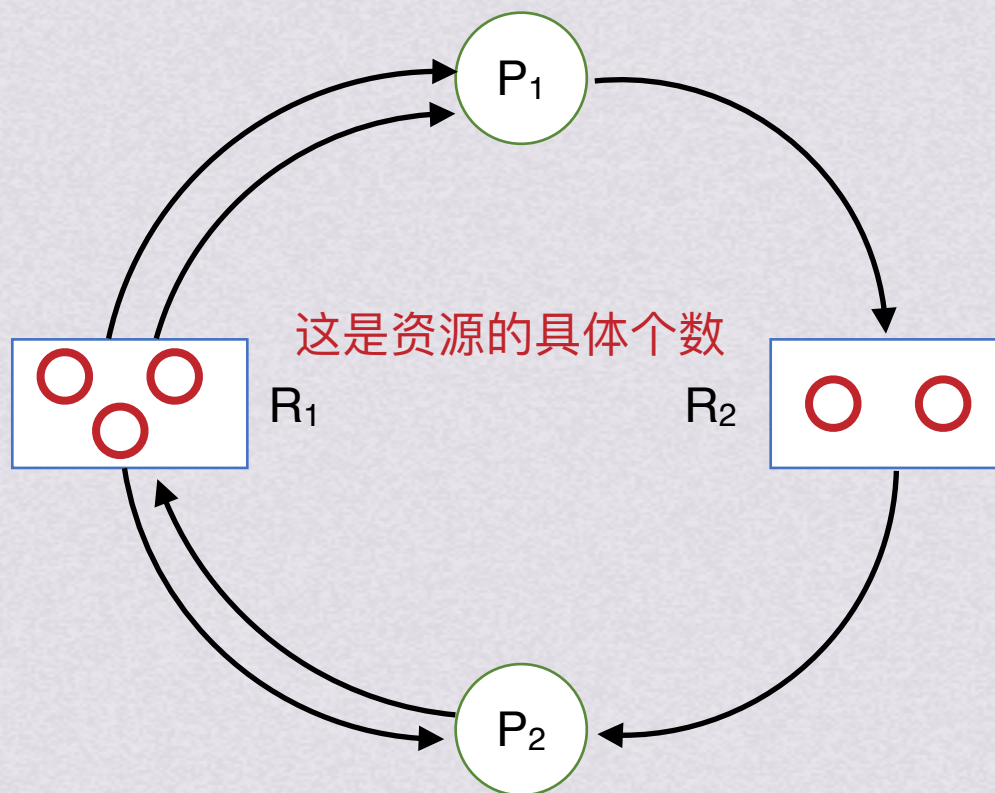


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

说人话：进程画圆圈，资源画方框里的圆圈



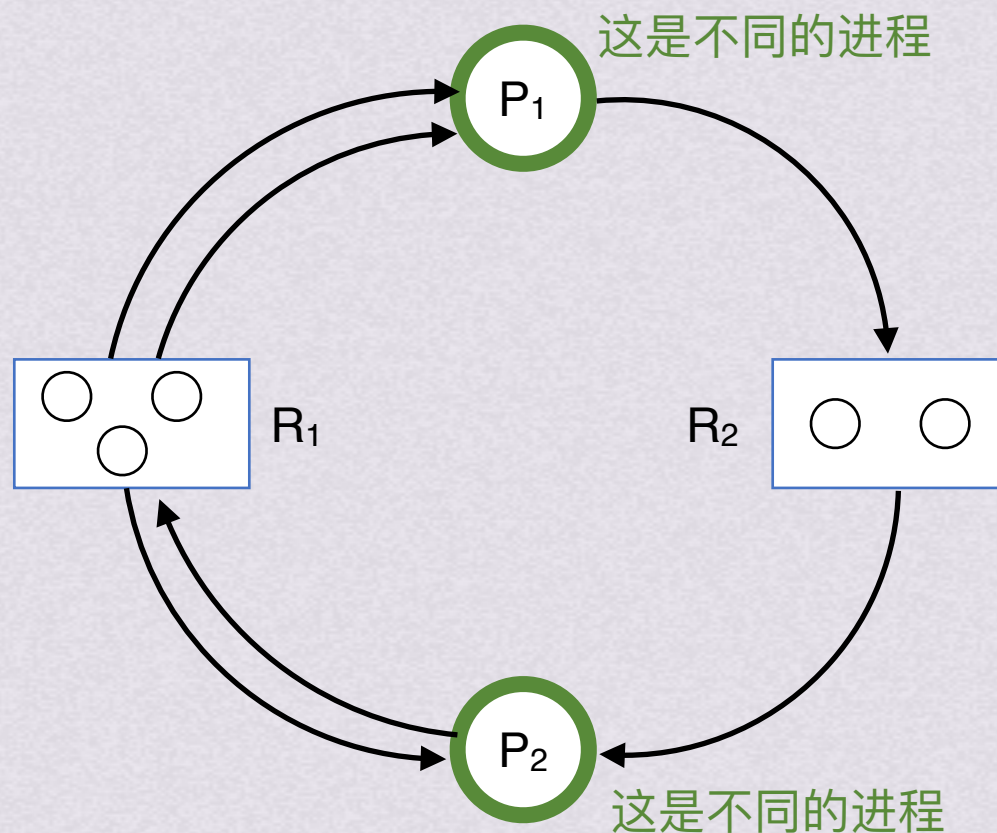


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 。把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边。

说人话：进程画圆圈，资源画方框里的圆圈



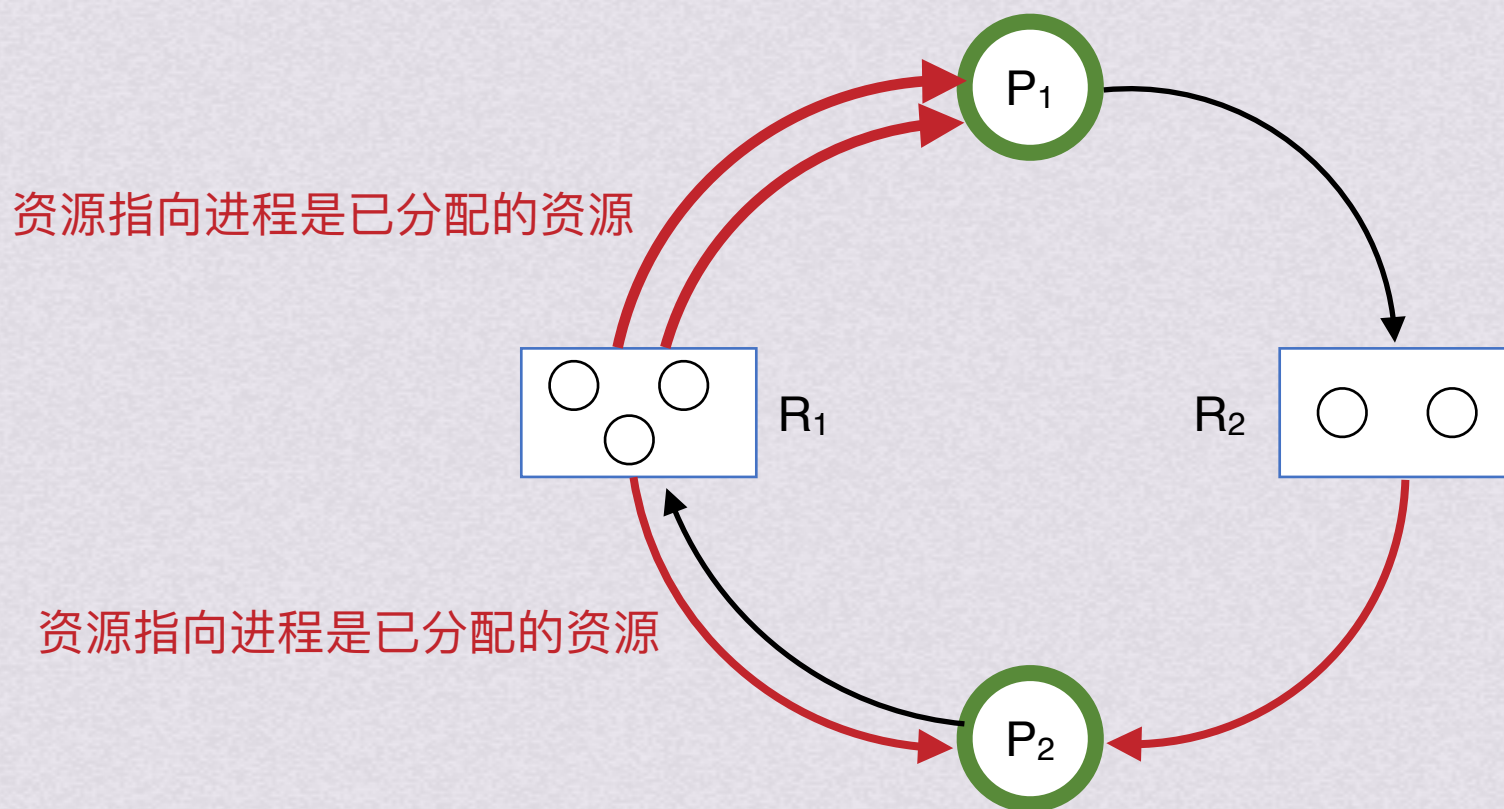


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

说人话：进程画圆圈，资源画方框里的圆圈



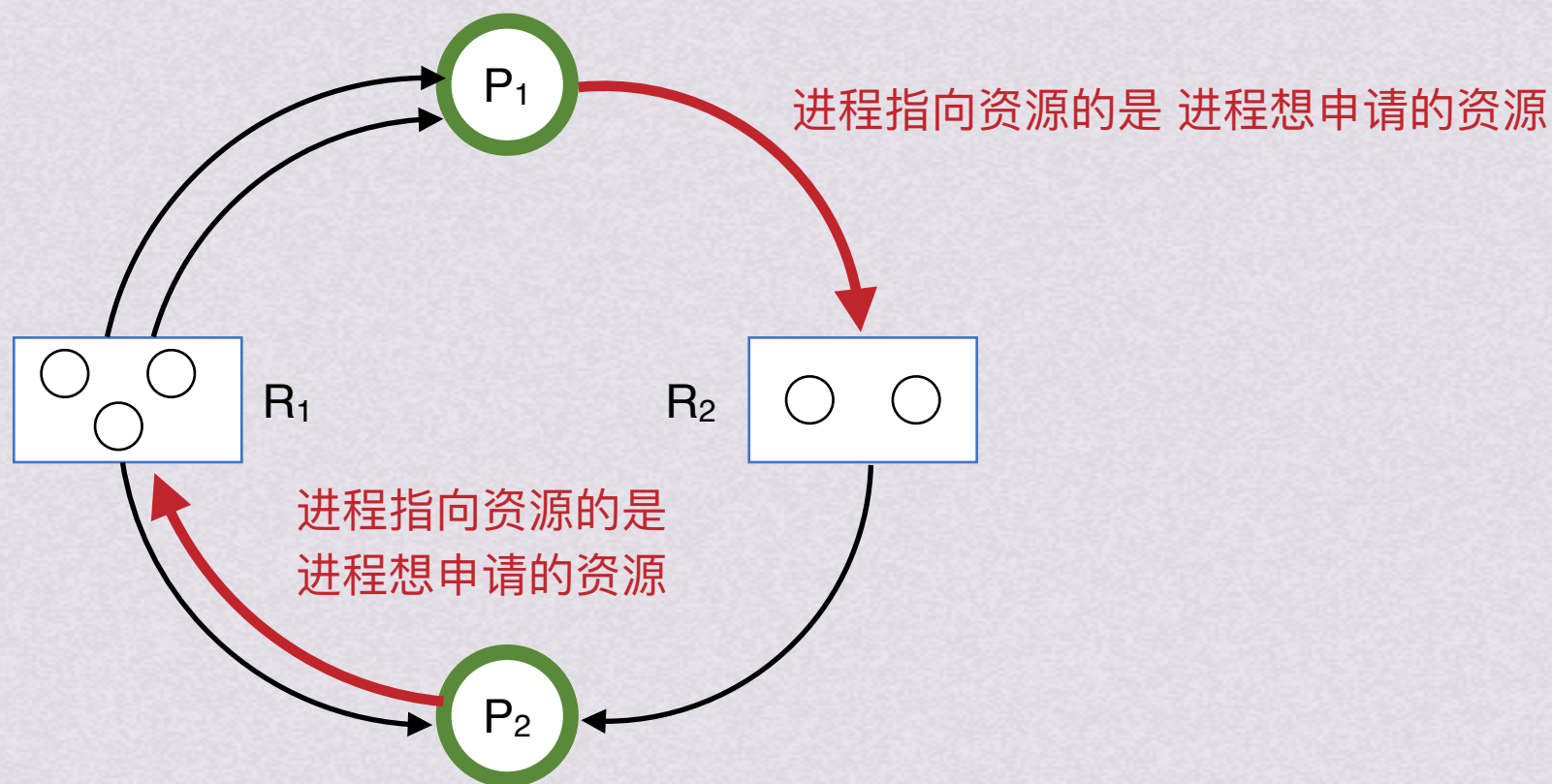


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

说人话：进程画圆圈，资源画方框里的圆圈



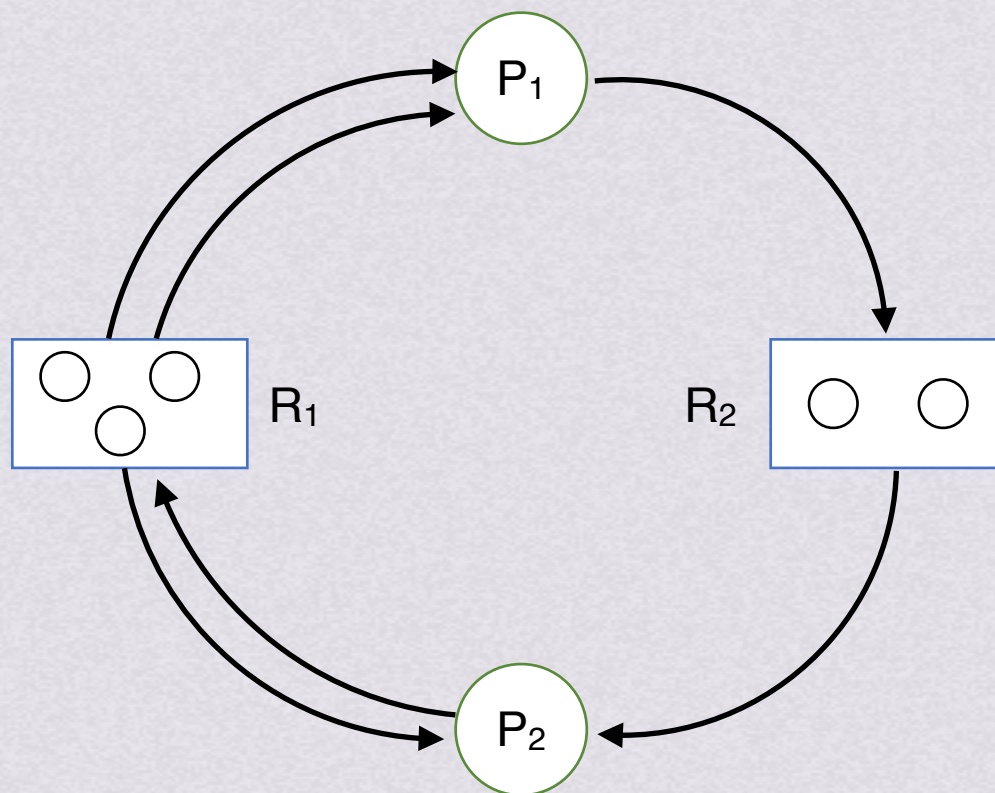


处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

说人话：进程画圆圈，资源画方框里的圆圈





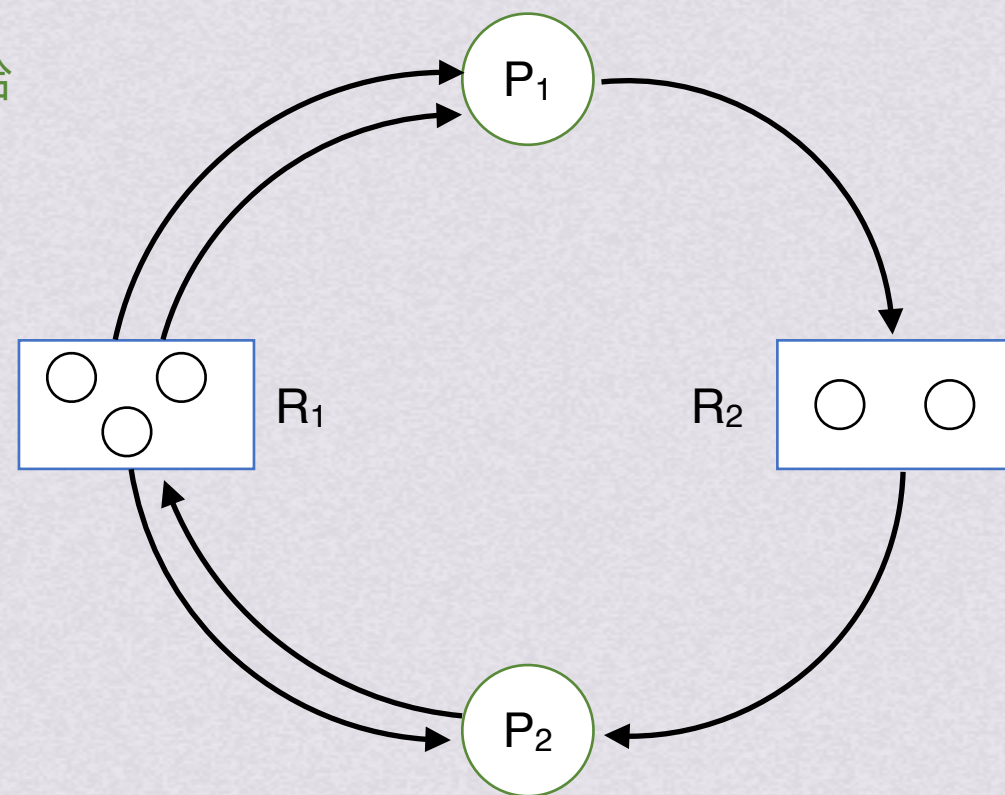
处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点 N 和一组边 E 组成的一个对偶 $G=(N,E)$ 把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R ，每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边； R 指向 P 的是资源分配边

资源分配图的化简：

1. 找出一个不阻塞也不独立的进程结点 P_i ，检查一切顺利的话能否给该进程所有需要的资源让其运行完毕。可以就释放其占用的所有资源，也就是消去他的所有请求边和分配边





处理方法

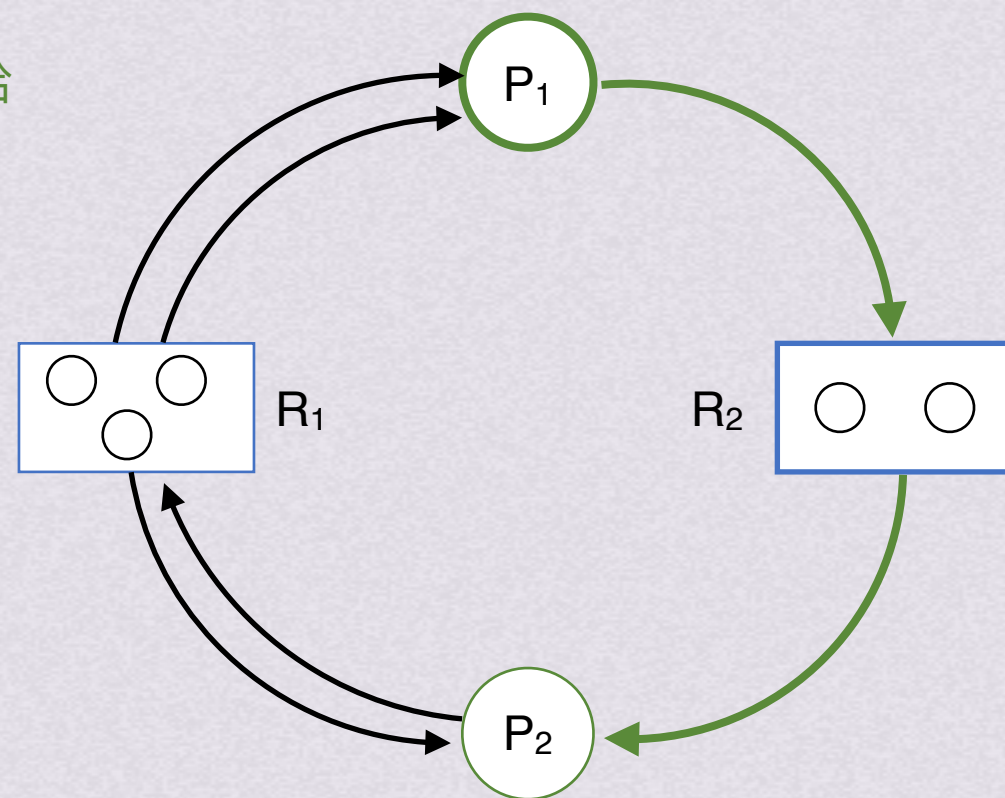
检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$ 把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

P_1 为例， R_2 已分配一个，还有一个正好给 P_1 用， P_1 拿到了所有所需资源

资源分配图的化简：

1. 找出一个不阻塞也不独立的进程结点 P_i ，检查一切顺利的话能否给该进程所有需要的资源让其运行完毕。可以就释放其占用的所有资源，也就是消去他的所有请求边和分配边





处理方法

检测死锁

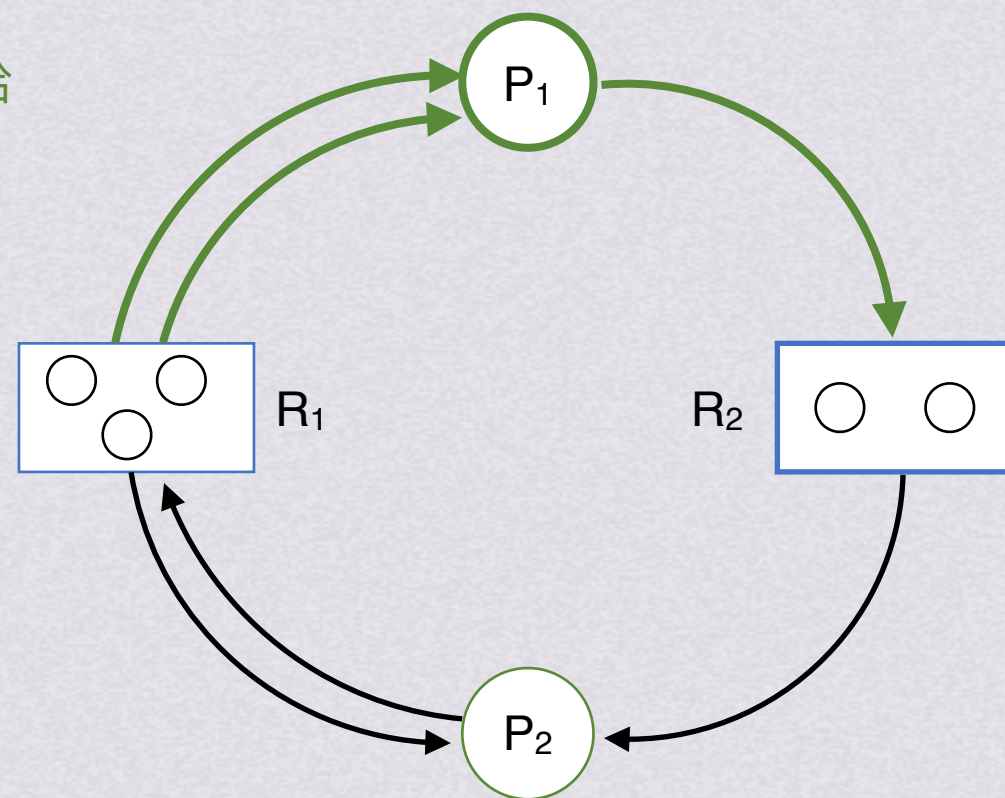
资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点 N 和一组边 E 组成的一个对偶 $G=(N,E)$ 把 N 分为两个互斥子集，一组进程节点 P 和一组资源节点 R ，每一个边都连着 P 和 R 中的一个结点， P 指向 R 的是资源请求边； R 指向 P 的是资源分配边

P_1 为例， R_2 已分配一个，还有一个正好给 P_1 用， P_1 拿到了所有所需资源
于是消去 P_1 的所有请求边和分配边

资源分配图的化简：

1. 找出一个不阻塞也不独立的进程结点 P_i ，检查一切顺利的话能否给该进程所有需要的资源让其运行完毕。可以就释放其占用的所有资源，也就是消去他的所有请求边和分配边

2. P_1 释放资源后就可以使 P_2 获得资源继续运行，把图中所有进程依次按照这种方法化简





处理方法

检测死锁

资源分配图：可利用资源分配图来描述系统死锁。资源分配图是由一组结点N和一组边E组成的一个对偶 $G=(N,E)$

把N分为两个互斥子集，一组进程节点P和一组资源节点R，每一个边都连着P和R中的一个结点，P指向R的是资源请求边；R指向P的是资源分配边

P_1 为例， R_2 已分配一个，还有一个正好给 P_1 用， P_1 拿到了所有所需资源

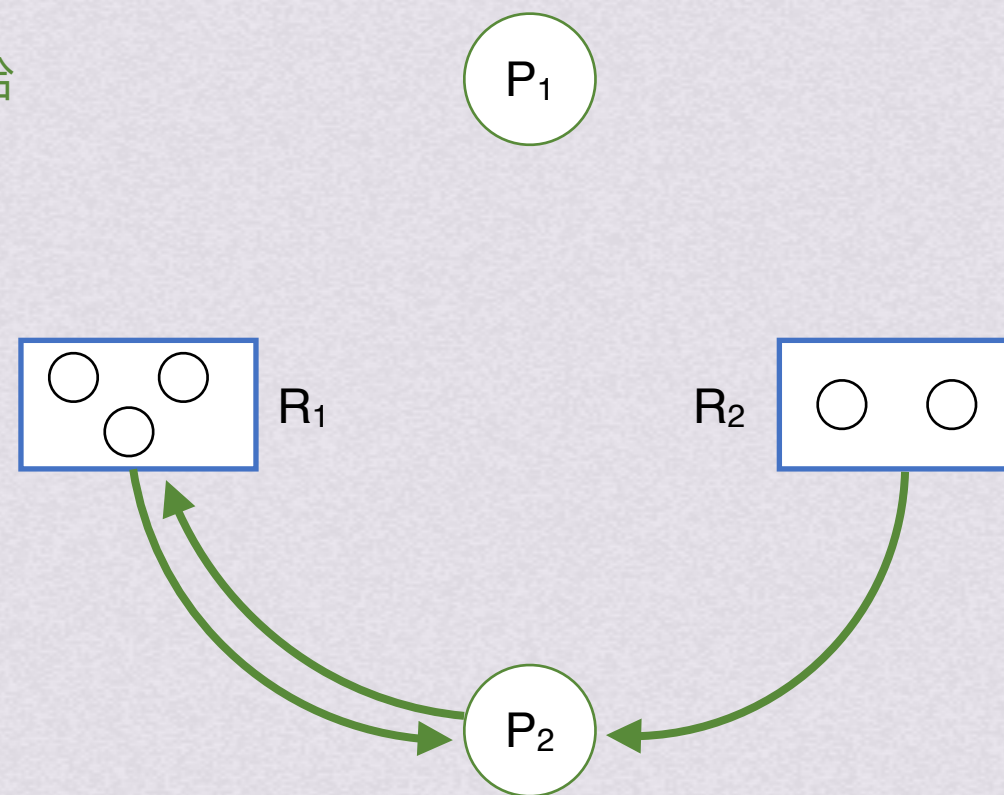
于是消去 P_1 的所有请求边和分配边

资源分配图的化简：

1. 找出一个不阻塞也不独立的进程结点 P_i ，检查一切顺利的话能否给该进程所有需要的资源让其运行完毕。可以就释放其占用的所有资源，也就是消去他的所有请求边和分配边

2. P_1 释放资源后就可以使 P_2 获得资源继续运行，把图中所有进程依次按照这种方法化简

3. 若能把图中所有的边都消除，则称该图是可完全简化的；如果该图是不可完全简化的，则此时处于死锁状态



P_1 不再占用 R_1 后， P_2 也能得到所有所需资源，于是消去 P_2 的所有请求边和分配边



检测死锁



解除死锁



处理方法

解除死锁

有三种常见的解除死锁的办法

1.资源剥夺法

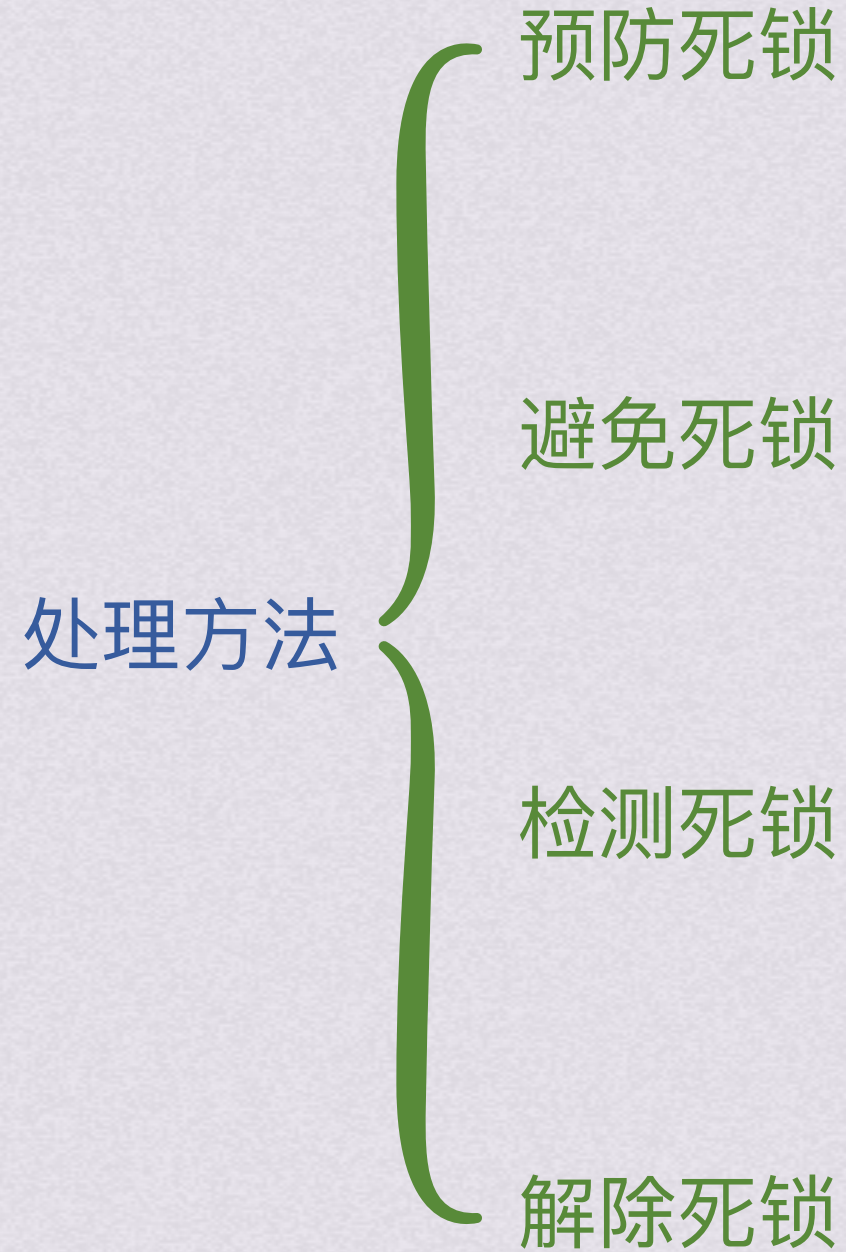
挂起一些死锁进程，抢占他的资源，分配给其他死锁进程

2.撤销进程法

强制撤销部分或者全部死锁进程并且剥夺资源。可以按照优先级和撤销代价来考虑撤销顺序，实现简单但是代价较大

3.进程回退法

让一个或多个死锁进程回退到足以回避死锁的地步，进程回退时自愿释放资源而不是被剥夺。但要求系统保持进程历史信息，设置还原点





处理方法

预防死锁

破坏4个必要条件

- 互斥条件
- 请求和保持条件
- 不可抢占条件
- 循环等待条件

避免死锁

银行家算法

- 安全性算法
- 可利用资源向量Available
- 最大需求矩阵Max
- 分配矩阵Allocation
- 需求矩阵Need

检测死锁

资源分配图

化简不了就死锁

解除死锁

进程全杀光，或者选择性杀掉

- 资源剥夺
- 撤销进程
- 进程回退